

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



ĐỒ ÁN CUỐI KỲ

**ĐỀ TÀI: XÂY DỰNG TRANG WEB THƯƠNG MẠI ĐIỆN
TỬ TÍCH HỢP CHATBOT TƯ VẤN**

Giảng viên hướng dẫn : TS. Kim Ngọc Bách

Sinh viên thực hiện : Phan Anh Minh Đức- B23DCCE022

Lớp : D23CQCE04-B

HÀ NỘI – 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn tới thầy Kim Ngọc Bách đã hướng dẫn, định hướng, góp ý và hỗ trợ em trong quá trình thực hiện đồ án. Cảm ơn thầy vì đã giúp em có cơ sở để triển khai đề tài theo hướng vừa có tính ứng dụng vừa có chiều sâu kỹ thuật.

Trong quá trình thực hiện, em đã cố gắng khảo sát bài toán, thiết kế kiến trúc, cài đặt hệ thống, kiểm thử các thành phần lõi và đánh giá hạn chế một cách trung thực. Do phạm vi thời gian và nguồn lực còn giới hạn, báo cáo vẫn còn nhiều thiếu sót. Em mong nhận được góp ý của thầy để tiếp tục hoàn thiện đề tài.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CẢM ƠN	2
DANH MỤC HÌNH	5
DANH MỤC BẢNG	5
DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ	6
PHẦN MỞ ĐẦU	8
CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ VÀ ĐỀ TÀI	9
1.1. Bối cảnh thương mại điện tử và sản phẩm công nghệ	9
1.2. Vấn đề người dùng gặp phải khi chọn sản phẩm	9
1.3. Hạn chế của bộ lọc truyền thống và chatbot tự do	9
1.4. Lý do cần AI Advisor có kiểm soát	10
1.5. Phương pháp thực hiện	10
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG	11
2.1. Tổng quan hệ thống thương mại điện tử	11
2.2. Tìm kiếm, lọc và hỗ trợ ra quyết định trong thương mại điện tử	12
2.3. Đặc điểm dữ liệu sản phẩm công nghệ	13
2.4. Large Language Models và giới hạn khi sử dụng trực tiếp	14
2.5. Retrieval-Augmented Generation và grounding dữ liệu	15
2.6. AI Agent, tool use và workflow nhiều bước	15
2.7. Harness Engineering cho AI Advisor	16
2.8. Guardrails, verifier và kiểm chứng câu trả lời	17
2.9. Evaluation cho AI Agent	18
2.10. Công nghệ sử dụng trong dự án	19
2.10.1. React, Vite và TypeScript	19
2.10.2. FastAPI, Pydantic và Uvicorn	20
2.10.3. Python cho xử lý catalog và AI workflow	21
2.10.4. Catalog dạng CSV và hướng mở rộng sang cơ sở dữ liệu	21
2.10.5. Pytest cho kiểm thử	22
2.10.6. Docker và Docker Compose	22
CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	23
3.1. Mô tả hệ thống bằng ngôn ngữ tự nhiên	23
3.2. Phân tích hệ thống	24
3.2.1. Yêu cầu hệ thống	24
3.2.2. Xác định tác nhân	26
3.2.3. Biểu đồ Use case tổng quát	26
3.2.4. Biểu đồ Use case chi tiết và kịch bản	27
3.3. Thiết kế hệ thống	37
3.3.1. Kiến trúc tổng thể	37
3.3.2. Thiết kế luồng xử lý catalog	37
3.3.3. Thiết kế luồng xử lý AI Advisor	38
3.4. Xây dựng biểu đồ lớp	38
3.4.1. Trích rút danh từ và lựa chọn danh từ làm thực thể	38
3.4.2. Xác định thuộc tính và phương thức cho các lớp	39
3.5. Kết luận chương	41
CHƯƠNG 4. CÀI ĐẶT VÀ KIỂM THỬ	42
4.1. Môi trường triển khai ứng dụng	42
4.2. Cấu trúc mã nguồn hệ thống	43
4.3. Cài đặt backend API	44
4.3.1. Các endpoint chính	44
4.3.2. Schema ChatRequest và ChatResponse	44

4.4. Cài đặt dữ liệu catalog và xử lý sản phẩm	45
4.4.1. Truy xuất và lọc catalog	47
4.5. Cài đặt frontend và giao diện hệ thống	47
4.5.1. Giao diện catalog sản phẩm	47
4.5.2. Giao diện ProductCard và chi tiết sản phẩm	48
4.5.3. Giao diện CopilotDrawer	49
4.6. Cài đặt AI Advisor và Harness Engineering	53
4.6.1. Pha perceive - nạp state và kiểm tra catalog revision	54
4.6.2. Pha plan - xác định intent và QueryFrame	55
4.6.3. Pha retrieve/execute - truy xuất catalog và sinh response	60
4.6.4. Evidence, Verifier và Response Contract	60
4.6.5. Trace và recovery	61
4.7. Kiểm thử hệ thống	62
4.7.1. Kiểm thử API thủ công bằng curl	62
4.7.2. Kiểm thử tự động backend	62
4.7.3. Kiểm thử frontend build	63
4.7.4. Kiểm thử kịch bản hội thoại	63
4.8. Đánh giá kết quả cài đặt	64
4.9. Kết luận chương	64
CHƯƠNG 5. KẾT LUẬN	65
5.1. Đánh giá kết quả đạt được	65
5.1.1. Kết quả về storefront và catalog sản phẩm	65
5.1.2. Kết quả về tích hợp AI Advisor	65
5.1.3. Kết quả về harness kiểm soát AI	66
5.1.4. Kết quả về kiểm thử và đánh giá	66
5.2. Ưu điểm của hệ thống	67
5.3. Hạn chế của hệ thống	67
5.4. Hướng phát triển trong tương lai	68
5.5. Kết luận chương	69
TÀI LIỆU THAM KHẢO	71

DANH MỤC HÌNH

Hình 3.1. Biểu đồ use case tổng quát của hệ thống AURA	27
Hình 3.2. Biểu đồ use case chi tiết nhóm chức năng Khách hàng	28
Hình 3.3. Biểu đồ use case chi tiết nhóm chức năng AI Advisor và harness	32
Hình 3.4. Biểu đồ use case chi tiết nhóm chức năng quản trị catalog và vận hành kỹ thuật	35
Hình 4.1. Cấu trúc mã nguồn hệ thống AURA	43
Hình 4.2. Pipeline xử lý dữ liệu catalog phục vụ hiển thị và AI grounding	46
Hình 4.3. Giao diện catalog sản phẩm	48
Hình 4.4. Giao diện ProductCard	49
Hình 4.5. Giao diện CopilotDrawer	51
Hình 4.6. Khung context trong CopilotDrawer	53
Hình 4.7. Luồng xử lý AI Advisor từ câu hỏi tới response contract	54
Hình 4.8. Minh họa câu hỏi cấu hình RAM/SSD và display_specs	56
Hình 4.9. Minh họa câu hỏi phân tích kỹ 1 mẫu máy	58
Hình 4.10. Minh họa câu hỏi có phù hợp cho văn phòng không	58
Hình 4.11. Minh họa câu hỏi so sánh cùng 1 tầm giá	59
Hình 4.12. Response Contract đồng bộ answer, product cards, verification và state	60
Hình 4.13. Minh họa decision_trace/harness trace	61

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Thuật ngữ	Giải thích gắn với dự án
AI Advisor	Module tư vấn bằng AI trong hệ thống AURA, nhận câu hỏi tiếng Việt tự nhiên và trả về câu trả lời có cấu trúc gắn với sản phẩm trong catalog.
LLM	Large Language Model - mô hình ngôn ngữ lớn có khả năng hiểu và sinh ngôn ngữ tự nhiên, nhưng không được xem là nguồn sự thật duy nhất trong hệ thống bán hàng.
RAG	Retrieval-Augmented Generation - cách kết hợp mô hình sinh với truy xuất dữ liệu bên ngoài để tăng grounding cho câu trả lời.
AI Agent	Thành phần AI có khả năng phân tích mục tiêu, lập kế hoạch, gọi công cụ, quan sát kết quả và sinh phản hồi.
Harness Engineering	Cách thiết kế lớp điều phối bao quanh AI agent: quản lý state, tool use, evidence, verifier, response contract, trace và recovery.
QueryFrame	Biểu diễn có cấu trúc của yêu cầu người dùng, gồm category, brand, budget, RAM, SSD, GPU, use_case, requested attributes.
AgentState	Trạng thái gọn của phiên tư vấn, lưu sản phẩm đang focus, candidate set, ngân sách, sở thích và intent gần nhất.
Evidence Ledger	Sổ bằng chứng ghi lại các facts, source_url, product_code, fetched_at và độ tin cậy được dùng để tạo câu trả lời.
Verifier	Bộ kiểm chứng output trước khi trả về frontend; chặn sai constraints, thiếu evidence, bìa field hoặc text-card mismatch.
Response Contract	Schema phản hồi giữa backend AI và frontend, gồm answer_text, related_products, display_specs, warnings, verification, state và trace.
Trace	Vết thực thi của một lượt hỏi đáp, ghi các phase như perception, planning, retrieval, verification, recovery và commit.
Product Card	Thẻ sản phẩm hiển thị ảnh, tên, giá, SKU

Thuật ngữ	Giải thích gắn với dự án
	và thông số nổi bật; được đồng bộ với câu trả lời AI bằng <code>display_specs</code> .
SKU/Product Code	Mã định danh sản phẩm trong catalog, dùng để truy xuất, hiển thị, resolve tham chiếu và kiểm tra <code>related_products</code> .
Catalog	Tập dữ liệu sản phẩm gồm tên, thương hiệu, giá, ảnh, <code>structured_specs</code> , <code>evidence facts</code> và <code>provenance</code> phục vụ cả UI lẫn AI <code>grounding</code> .

Bảng 0.1. Danh mục từ viết tắt và thuật ngữ

PHẦN MỞ ĐẦU

Sự phát triển của thương mại điện tử khiến người dùng có thể tiếp cận nhiều lựa chọn hơn, nhưng cũng làm quá trình ra quyết định trở nên khó khăn hơn. Với sản phẩm công nghệ, người mua không chỉ so sánh giá mà còn phải hiểu cấu hình, hiệu năng, nhu cầu sử dụng, khả năng nâng cấp, thời điểm giá và độ phù hợp giữa nhiều biến thể sản phẩm. Đây là bối cảnh khiến một hệ thống tư vấn dựa trên ngôn ngữ tự nhiên trở nên có ý nghĩa.

Xuất phát từ thực tế đó, cùng với những kiến thức được học và nghiên cứu trong quá trình học, em đã chọn đề tài là xây dựng 1 hệ thống thương mại điện tử. Tuy nhiên, so với một website thương mại điện tử thông thường, đề án AURA đặt trọng tâm vào câu hỏi kỹ thuật: làm thế nào để tích hợp AI Advisor vào hệ thống e-commerce mà không biến nó thành một chatbot tự do khó kiểm chứng.

Luận điểm xuyên suốt của báo cáo là: AURA không chỉ là một website hiển thị catalog, cũng không chỉ là chatbot hỏi đáp. Đề án xây dựng một hệ thống e-commerce có AI Advisor được đặt trong Harness Engineering để kiểm soát state, truy xuất catalog, bằng chứng, kiểm chứng, response contract và trace.

Ngoài phần mở đầu và kết luận, đề án được kết cấu thành 5 chương chính:

Chương 1: Tổng quan về hệ thống thương mại điện tử và đề tài: Chương 1 giới thiệu bối cảnh của thương mại điện tử, tổng quan về hệ thống, vấn đề người dùng gặp phải khi chọn sản phẩm, hạn chế của các sản phẩm truyền thống và phương pháp thực hiện.

Chương 2: Đưa cơ sở lý thuyết dẫn tới thiết kế: Chương 2 trình bày về cơ sở lý thuyết và công nghệ được đưa vào sử dụng để xây dựng hệ thống

Chương 3: Phân tích thiết kế hệ thống: Chương 3 trình bày quá trình phân tích, thiết kế hệ thống bao gồm: các yêu cầu đối với hệ thống, các tác nhân trong hệ thống, các usecase và kịch bản usecase sử dụng..

Chương 4: Cài đặt và kiểm thử: Chương 4 trình bày môi trường triển khai hệ thống và giao diện của hệ thống.

Chương 5: Đánh giá kết quả và Hướng phát triển: Chương 5 trình bày về kết quả đạt được, ưu điểm và hạn chế của hệ thống, và hướng phát triển trong tương lai.

CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG THƯƠNG MẠI ĐIỆN TỬ VÀ ĐỀ TÀI

1.1. Bối cảnh thương mại điện tử và sản phẩm công nghệ

Thương mại điện tử tạo điều kiện để người dùng tiếp cận số lượng sản phẩm lớn hơn nhiều so với mua sắm truyền thống. Đối với sản phẩm công nghệ, lợi ích này đi kèm với một khó khăn đáng kể: mỗi sản phẩm thường có nhiều thuộc tính kỹ thuật như CPU, RAM, SSD, GPU, màn hình, camera, pin, trọng lượng, giá bán, bảo hành và tình trạng hàng. Người dùng phổ thông không phải lúc nào cũng hiểu thông số nào quan trọng với nhu cầu của mình.

Một website e-commerce thông thường thường giải quyết bài toán này bằng danh mục, thanh tìm kiếm, bộ lọc và trang chi tiết. Các chức năng này cần thiết nhưng chưa đủ khi người dùng diễn đạt nhu cầu theo mục tiêu, ví dụ “laptop học lập trình dưới 20 triệu”, “điện thoại chụp ảnh đẹp nhưng pin ổn”, hoặc “mẫu vừa rồi có lựa chọn nào nhẹ hơn không”. Những câu hỏi như vậy cần một lớp diễn giải nhu cầu, chuyển từ ngôn ngữ tự nhiên sang ràng buộc sản phẩm có thể truy xuất.

1.2. Vấn đề người dùng gặp phải khi chọn sản phẩm

Khó khăn của người dùng không chỉ là thiếu thông tin, mà là quá nhiều thông tin nhưng khó đặt vào ngữ cảnh. Hai laptop cùng RAM 16GB có thể khác nhau về CPU, GPU, trọng lượng, màn hình và mức giá. Hai điện thoại cùng tầm giá có thể khác nhau về camera, pin, màn hình hoặc thương hiệu. Người dùng cần một hệ thống giúp diễn giải thông số thành khuyến nghị theo nhu cầu, đồng thời vẫn cho phép đối chiếu với dữ liệu sản phẩm cụ thể.

Nếu không có tư vấn, người dùng phải tự đọc nhiều product cards, tự lọc và tự so sánh. Điều này dễ tạo ra quyết định dựa trên một vài thông số nổi bật nhưng bỏ qua ràng buộc quan trọng. Vì vậy, bài toán thực tế của đề án là hỗ trợ ra quyết định chọn sản phẩm công nghệ dựa trên catalog thật, không chỉ hiển thị dữ liệu.

1.3. Hạn chế của bộ lọc truyền thống và chatbot tự do

Bộ lọc truyền thống có ưu điểm là rõ ràng, dễ kiểm soát và nhất quán với dữ liệu catalog. Tuy nhiên, nó yêu cầu người dùng biết trước các trường kỹ thuật cần lọc. Ngược lại, chatbot tự do giúp người dùng hỏi bằng ngôn ngữ tự nhiên nhưng có rủi ro cao nếu để mô hình trả lời dựa trên trí nhớ hoặc suy luận không được kiểm chứng. Trong hệ thống bán hàng, một câu trả lời sai giá, sai SKU hoặc sai cấu hình có thể làm người dùng ra quyết định sai.

Do đó, hướng tiếp cận của AURA không phải “thay bộ lọc bằng chatbot”, mà là kết hợp hai hướng: giữ nền e-commerce có catalog và product cards, đồng thời bổ sung AI Advisor có cơ chế grounding, verifier và response contract. AI hỗ trợ diễn giải và tư vấn, nhưng kết quả phải quay về danh sách sản phẩm cụ thể trong catalog.

1.4. Lý do cần AI Advisor có kiểm soát

AI Advisor có kiểm soát là cần thiết vì câu trả lời trong e-commerce cần đúng dữ liệu, đúng ngữ cảnh và đồng bộ giao diện. Khi người dùng hỏi “mẫu đó”, hệ thống phải biết sản phẩm đang được nhắc tới. Khi người dùng hỏi RAM/SSD, product card phải ưu tiên hiển thị RAM/SSD. Khi catalog thiếu thông tin pin hoặc trọng lượng, AI phải công bố thiếu dữ liệu thay vì bịa. Các yêu cầu này không thể được đảm bảo chỉ bằng prompt chung.

Vì vậy, đồ án đặt AI Advisor trong harness engineering. Harness là lớp điều phối quanh agent, kiểm soát đầu vào, state, query frame, retrieval, evidence, verifier, recovery, response contract và trace. Đây là điểm khác biệt chính giữa AURA và một chatbot/RAG thông thường.

1.5. Phương pháp thực hiện

1. Khảo sát bài toán tư vấn sản phẩm công nghệ và xác định hạn chế của filter/chatbot tự do.
2. Thiết kế kiến trúc frontend, backend, catalog service và AI Advisor trong cùng trải nghiệm e-commerce.
3. Chuẩn hóa dữ liệu catalog thành các trường có cấu trúc và facts có thể dùng làm evidence.
4. Thiết kế harness gồm state, query frame, planner, tool/retrieval layer, evidence, verifier, response contract và trace.
5. Cài đặt prototype và kiểm thử theo nhiều lớp: unit, integration, scenario, trace-level eval và browser QA đề xuất.
6. Đánh giá trung thực các điểm đã chứng minh, điểm còn thiếu và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

Chương này trình bày các cơ sở lý thuyết và công nghệ nền tảng được sử dụng trong quá trình xây dựng hệ thống AURA AI sales advisor. Khác với một website thương mại điện tử thông thường chỉ tập trung vào hiển thị sản phẩm, tìm kiếm và thao tác mua hàng, AURA hướng đến bài toán hỗ trợ người dùng lựa chọn sản phẩm công nghệ thông qua một AI Advisor được kiểm soát bằng dữ liệu catalog thật.

Nội dung chương được tổ chức theo hai nhóm chính. Nhóm thứ nhất trình bày các cơ sở lý thuyết liên quan đến hệ thống thương mại điện tử, dữ liệu sản phẩm công nghệ, tìm kiếm - lọc sản phẩm và hỗ trợ ra quyết định. Nhóm thứ hai trình bày các nền tảng liên quan đến trí tuệ nhân tạo trong hệ thống, bao gồm Large Language Models, Retrieval-Augmented Generation, AI Agent, harness engineering, guardrails, verifier và đánh giá AI agent. Cuối chương là phần mô tả các công nghệ được sử dụng để triển khai hệ thống.

Mục tiêu của chương không chỉ là liệt kê công nghệ, mà còn làm rõ vì sao từng nền tảng lý thuyết và từng công nghệ được lựa chọn, đồng thời chỉ ra cách chúng dẫn tới các quyết định phân tích, thiết kế và cài đặt ở các chương sau.

2.1. Tổng quan hệ thống thương mại điện tử

Thương mại điện tử là hình thức thực hiện các hoạt động mua bán, giới thiệu sản phẩm, trao đổi thông tin và hỗ trợ khách hàng thông qua môi trường Internet. Một hệ thống thương mại điện tử thường bao gồm các chức năng cơ bản như hiển thị danh mục sản phẩm, tìm kiếm và lọc sản phẩm, xem chi tiết sản phẩm, quản lý tài khoản, quản lý giỏ hàng, đặt hàng, thanh toán, theo dõi đơn hàng và hỗ trợ khách hàng.

Trong phạm vi đề án AURA, hệ thống thương mại điện tử không được triển khai theo hướng đầy đủ tất cả nghiệp vụ bán hàng như thanh toán, vận chuyển hay quản lý kho ở quy mô production. Thay vào đó, đề tài tập trung vào phần cốt lõi phục vụ bài toán tư vấn sản phẩm công nghệ, bao gồm: catalog sản phẩm, tìm kiếm/lọc sản phẩm, product cards, giao diện xem sản phẩm và AI Advisor hỗ trợ lựa chọn sản phẩm.

Đối với các sản phẩm công nghệ như laptop và điện thoại, người dùng thường không chỉ quan tâm đến tên sản phẩm hay giá bán, mà còn cần hiểu các thông số như CPU, RAM, SSD, GPU, màn hình, pin, trọng lượng, thương hiệu và mục đích sử dụng. Vì vậy, catalog trong hệ thống không chỉ đóng vai trò là danh sách sản phẩm để hiển thị, mà còn là nguồn dữ liệu nền để AI Advisor truy xuất, kiểm chứng và tạo câu trả lời.

Trong hệ thống AURA, các thành phần giao diện như ProductGrid, ProductCard và CopilotDrawer được thiết kế để kết hợp trải nghiệm thương mại điện tử truyền thống với trải nghiệm tư vấn bằng ngôn ngữ tự nhiên. Người dùng vẫn có thể duyệt sản phẩm theo cách thông thường, đồng thời có thể đặt câu hỏi như “Cho tôi laptop Dell dưới 20 triệu để học lập trình” hoặc “Hai mẫu này khác nhau điểm nào?”. Câu trả lời của AI

Advisor không chỉ hiển thị dưới dạng văn bản, mà còn gắn với các product cards tương ứng để người dùng có thể đối chiếu.

Như vậy, nền tảng thương mại điện tử trong đồ án giữ vai trò quan trọng ở ba khía cạnh. Thứ nhất, nó cung cấp bối cảnh nghiệp vụ thực tế cho bài toán tư vấn sản phẩm. Thứ hai, nó cung cấp dữ liệu catalog có cấu trúc để AI Advisor có căn cứ phản hồi. Thứ ba, nó tạo ra giao diện kiểm chứng trực quan, giúp người dùng không phải tin hoàn toàn vào câu trả lời dạng văn bản của AI.

Từ cơ sở đó, ở chương phân tích và thiết kế hệ thống, AURA được thiết kế không phải như một chatbot độc lập, mà là một hệ thống e-commerce tích hợp AI Advisor. Đây là điểm khác biệt quan trọng của đề tài so với các chatbot demo chỉ trả lời tự do mà không ràng buộc với sản phẩm thật.

2.2. Tìm kiếm, lọc và hỗ trợ ra quyết định trong thương mại điện tử

Trong hệ thống thương mại điện tử, tìm kiếm và lọc sản phẩm là hai chức năng quan trọng giúp người dùng thu hẹp không gian lựa chọn. Với catalog nhỏ, người dùng có thể duyệt thủ công từng sản phẩm. Tuy nhiên, khi số lượng sản phẩm tăng lên, việc duyệt thủ công trở nên kém hiệu quả. Người dùng cần các cơ chế như tìm kiếm theo từ khóa, lọc theo danh mục, lọc theo thương hiệu, lọc theo mức giá, sắp xếp theo độ phù hợp hoặc so sánh giữa các sản phẩm.

Các nghiên cứu và hướng dẫn về trải nghiệm người dùng trong thương mại điện tử cho thấy faceted search, tức tìm kiếm kết hợp nhiều bộ lọc, có vai trò quan trọng trong việc giúp người dùng điều hướng catalog lớn [1]. Đối với sản phẩm công nghệ, faceted search càng cần thiết vì mỗi sản phẩm có nhiều thuộc tính kỹ thuật khác nhau. Một người dùng khi mua laptop có thể quan tâm đồng thời đến thương hiệu, giá, CPU, RAM, SSD, GPU, màn hình và mục đích sử dụng.

Tuy nhiên, bộ lọc truyền thống vẫn tồn tại một hạn chế lớn: người dùng phải biết trước mình cần lọc theo trường nào. Trong thực tế, nhiều người dùng không diễn đạt nhu cầu theo cấu trúc dữ liệu của hệ thống. Họ thường nói theo mục tiêu sử dụng, ví dụ: “máy học lập trình ổn”, “laptop văn phòng nhẹ”, “máy chơi game tầm giá 25 triệu”, “điện thoại chụp ảnh đẹp”. Những câu hỏi này không khớp trực tiếp với một trường dữ liệu duy nhất trong catalog, mà cần được phân tích thành nhiều ràng buộc khác nhau.

Vì vậy, trong AURA, khái niệm **QueryFrame** được sử dụng như một lớp trung gian giữa ngôn ngữ tự nhiên và bộ lọc sản phẩm. QueryFrame có thể hiểu là một “bộ lọc được AI hóa”. Thay vì yêu cầu người dùng tự chọn từng bộ lọc, hệ thống phân tích câu hỏi tự nhiên để rút ra các thành phần như danh mục sản phẩm, thương hiệu, khoảng giá, mục đích sử dụng, yêu cầu về CPU, GPU, RAM, SSD hoặc các thuộc tính cần hỏi thêm.

Ví dụ, câu hỏi “Cho tôi laptop Dell dưới 20 triệu để học lập trình” có thể được chuyển thành QueryFrame gồm: `category = Laptop`, `brand = Dell`, `budget_max = 20.000.000`,

use_case = programming. Từ QueryFrame này, hệ thống có thể truy xuất catalog một cách có kiểm soát thay vì để mô hình ngôn ngữ tự sinh câu trả lời.

Cách tiếp cận này giúp hệ thống kết hợp được ưu điểm của giao diện tìm kiếm truyền thống và tư vấn bằng ngôn ngữ tự nhiên. Bộ lọc truyền thống đảm bảo tính xác định và dễ kiểm chứng, trong khi AI Advisor giúp người dùng diễn đạt nhu cầu tự nhiên hơn. Đây là một cơ sở quan trọng dẫn đến thiết kế retrieval, product resolver và response contract trong các chương tiếp theo.

2.3. Đặc điểm dữ liệu sản phẩm công nghệ

Dữ liệu sản phẩm công nghệ có nhiều đặc điểm phức tạp hơn so với các loại sản phẩm đơn giản. Một sản phẩm laptop hoặc điện thoại thường có tên dài, nhiều mã model, nhiều biến thể cấu hình và nhiều thuộc tính kỹ thuật. Các thông tin quan trọng có thể nằm trong các trường khác nhau như tên sản phẩm, mô tả, bảng thông số, dữ liệu giá hoặc các đoạn mô tả bán hàng.

Một sản phẩm công nghệ thường bao gồm các nhóm thông tin sau:

- Thông tin định danh: mã sản phẩm, SKU, tên sản phẩm, thương hiệu, danh mục.
- Thông tin thương mại: giá, trạng thái hiển thị, nguồn dữ liệu, thời điểm cập nhật.
- Thông tin kỹ thuật: CPU, GPU, RAM, SSD, màn hình, pin, trọng lượng, camera.
- Thông tin mô tả: mô tả ngắn, mô tả dài, ưu điểm nổi bật, ngữ cảnh sử dụng.
- Thông tin nguồn: URL nguồn, thời điểm lấy dữ liệu, provenance của từng thông số.

Vấn đề đặt ra là dữ liệu này không phải lúc nào cũng đầy đủ và đồng nhất. Một số sản phẩm có thông tin CPU rõ ràng nhưng thiếu trọng lượng; một số sản phẩm có giá dạng chuỗi văn bản; một số thông số nằm lẫn trong mô tả; một số sản phẩm có nhiều biến thể gần giống nhau. Nếu hệ thống AI không nhận biết được trường nào có dữ liệu và trường nào bị thiếu, nó có thể tạo ra các nhận định không có căn cứ.

Trong bài toán tư vấn sản phẩm, lỗi dữ liệu có thể dẫn đến lỗi tư vấn. Ví dụ, nếu catalog không có thông tin pin nhưng AI Advisor vẫn khẳng định “máy này pin rất tốt”, câu trả lời đó trở thành hallucination theo nghĩa nghiệp vụ. Tương tự, nếu hệ thống nhầm mã SKU giữa hai sản phẩm gần giống nhau, người dùng có thể nhận được gợi ý sai sản phẩm.

Vì vậy, dữ liệu sản phẩm trong AURA cần được xử lý theo hướng có cấu trúc và có thể kiểm chứng. Các thông tin từ catalog cần được chuẩn hóa thành các trường như product_code, category, brand, price, price_value, specs, display_specs, source_url, fetched_at và spec_provenance. Những trường này không chỉ phục vụ hiển thị giao diện, mà còn là cơ sở cho retrieval, evidence ledger và verifier.

Trong thiết kế hệ thống, phần catalog pipeline cần thực hiện các bước như làm sạch dữ liệu, chuẩn hóa giá, trích xuất thông số, tạo facts cho truy vấn, lưu nguồn tham chiếu và đánh dấu missing fields. Với cách làm này, AI Advisor không chỉ “đọc” catalog, mà còn biết giới hạn của catalog. Khi thiếu dữ liệu, hệ thống có thể trả lời theo hướng thận trọng, ví dụ “catalog hiện chưa có thông tin trọng lượng của sản phẩm này”, thay vì tự suy đoán.

Đây là điểm quan trọng giúp AURA khác với một chatbot bán hàng thông thường. Trọng tâm của hệ thống không phải là sinh ra câu trả lời nghe thuyết phục, mà là sinh ra câu trả lời có căn cứ từ catalog thật.

2.4. Large Language Models và giới hạn khi sử dụng trực tiếp

Large Language Models là các mô hình ngôn ngữ quy mô lớn có khả năng hiểu, sinh và biến đổi văn bản tự nhiên. Sự phát triển của kiến trúc Transformer đã tạo nền tảng cho nhiều mô hình ngôn ngữ hiện đại nhờ khả năng học quan hệ ngữ cảnh giữa các token trong chuỗi văn bản [2]. Nhờ đó, LLM có thể thực hiện nhiều tác vụ như trả lời câu hỏi, tóm tắt, viết lại văn bản, phân loại ý định và hỗ trợ hội thoại.

Trong hệ thống tư vấn sản phẩm, LLM có một số ưu điểm rõ rệt. Mô hình có thể hiểu cách diễn đạt tự nhiên của người dùng, xử lý câu hỏi không theo khuôn mẫu, duy trì hội thoại nhiều lượt và tạo phản hồi mềm mại hơn so với các hệ thống rule-based thuần túy. Ví dụ, người dùng có thể hỏi “mẫu đó có hợp để học lập trình không?” hoặc “có mẫu nào rẻ hơn mà vẫn ổn không?”. Đây là các câu hỏi phụ thuộc vào ngữ cảnh và khó xử lý nếu chỉ dùng tìm kiếm từ khóa.

Tuy nhiên, LLM cũng có giới hạn quan trọng. Thứ nhất, LLM không phải là cơ sở dữ liệu sản phẩm. Mô hình không tự biết catalog hiện tại của hệ thống, giá mới nhất, tình trạng sản phẩm hoặc mã SKU đang được hiển thị trên giao diện. Thứ hai, LLM có thể sinh ra thông tin không có trong dữ liệu, còn gọi là hallucination. Thứ ba, LLM có thể trả lời quá tự tin ngay cả khi thiếu bằng chứng. Thứ tư, nếu không được kiểm soát, LLM có thể tạo ra câu trả lời không đồng bộ với product cards trên giao diện.

Trong lĩnh vực thương mại điện tử, các lỗi này có tác động nghiêm trọng hơn so với hội thoại thông thường. Một câu trả lời sai về giá, cấu hình, khả năng chơi game hoặc khả năng nâng cấp có thể làm người dùng hiểu nhầm và đưa ra quyết định mua hàng sai. Vì vậy, việc sử dụng LLM trực tiếp để tư vấn sản phẩm là không đủ an toàn.

AURA giải quyết vấn đề này bằng cách không để LLM hoạt động tự do. LLM chỉ là một thành phần trong pipeline được kiểm soát. Trước khi câu trả lời được tạo ra, hệ thống cần phân tích ý định, tạo QueryFrame, truy xuất catalog và xây dựng evidence. Sau khi câu trả lời được tạo ra, hệ thống cần verifier kiểm tra tính nhất quán giữa nội dung trả lời, danh sách sản phẩm liên quan và dữ liệu catalog.

Cách tiếp cận này giúp tận dụng khả năng ngôn ngữ tự nhiên của LLM nhưng vẫn giới hạn phạm vi trả lời trong dữ liệu có căn cứ. Đây cũng là cơ sở cho thiết kế AI Advisor theo hướng agent có harness thay vì chatbot sinh văn bản thuần túy.

2.5. Retrieval-Augmented Generation và grounding dữ liệu

Retrieval-Augmented Generation là kỹ thuật kết hợp mô hình sinh ngôn ngữ với cơ chế truy xuất thông tin từ nguồn dữ liệu bên ngoài [3]. Thay vì để mô hình chỉ dựa vào tri thức đã học trong quá trình huấn luyện, hệ thống sẽ truy xuất các tài liệu hoặc bản ghi liên quan, sau đó cung cấp chúng làm ngữ cảnh cho mô hình sinh câu trả lời.

Trong AURA, nguồn dữ liệu bên ngoài không phải là các tài liệu văn bản dài, mà là catalog sản phẩm và các facts được trích xuất từ dữ liệu sản phẩm. Khi người dùng đặt câu hỏi, hệ thống cần tìm ra những sản phẩm phù hợp nhất, lấy các thông tin liên quan như giá, thương hiệu, cấu hình, ưu điểm và các trường còn thiếu, sau đó sử dụng những thông tin này để tạo phản hồi.

RAG mang lại một lợi ích quan trọng là giúp câu trả lời gắn với dữ liệu cụ thể. Với câu hỏi “laptop nào dưới 20 triệu phù hợp học lập trình”, hệ thống không nên trả lời dựa trên kiến thức chung về laptop, mà cần truy xuất những sản phẩm thật trong catalog thỏa mãn điều kiện về giá và mục đích sử dụng. Nhờ đó, câu trả lời có thể đi kèm product cards và mã sản phẩm cụ thể.

Tuy nhiên, RAG thông thường chưa đủ để đảm bảo độ tin cậy trong bài toán tư vấn bán hàng. Có ba vấn đề cần xử lý thêm. Thứ nhất, retrieval có thể trả về candidate chưa thật sự phù hợp nếu truy vấn tự nhiên được hiểu sai. Thứ hai, dữ liệu trả về có thể thiếu một số thông tin mà người dùng đang hỏi. Thứ ba, mô hình sinh vẫn có thể diễn giải quá mức hoặc thêm nhận định không nằm trong evidence.

Do đó, AURA bổ sung thêm hai thành phần sau retrieval: **Evidence Ledger** và **Verifier**. Evidence Ledger ghi nhận các dữ kiện được phép sử dụng trong câu trả lời, ví dụ mã sản phẩm, giá, thương hiệu, thông số kỹ thuật và nguồn dữ liệu. Verifier kiểm tra xem câu trả lời cuối cùng có vượt quá dữ kiện này hay không. Nếu hệ thống không có thông tin về pin, trọng lượng hoặc khả năng nâng cấp, verifier cần phát hiện và yêu cầu phản hồi an toàn hơn.

Như vậy, grounding trong AURA không chỉ là đưa dữ liệu vào prompt, mà là một quy trình kiểm soát nhiều bước. Retrieval giúp tìm dữ liệu, Evidence Ledger giúp quản lý dữ kiện, Verifier giúp kiểm tra câu trả lời, còn Response Contract giúp đồng bộ kết quả giữa backend và frontend. Đây là nền tảng để hệ thống AI Advisor hoạt động đáng tin cậy hơn trong ngữ cảnh e-commerce.

2.6. AI Agent, tool use và workflow nhiều bước

AI Agent là hệ thống sử dụng mô hình AI để thực hiện nhiệm vụ thông qua nhiều bước xử lý, thay vì chỉ sinh ra một câu trả lời đơn lẻ. Một agent thường có khả năng tiếp nhận yêu cầu, phân tích mục tiêu, lập kế hoạch, gọi công cụ, quan sát kết quả, cập nhật trạng thái và tạo phản hồi cuối cùng. Phương pháp ReAct là một ví dụ tiêu biểu cho việc kết hợp reasoning và action, trong đó mô hình vừa suy luận vừa thực hiện hành động với môi trường hoặc công cụ bên ngoài [4].

Trong các hệ thống hiện đại, tool use hoặc function calling cho phép mô hình gọi các công cụ cụ thể như tìm kiếm sản phẩm, truy xuất thông tin sản phẩm, lấy trường dữ liệu, kiểm tra đơn hàng hoặc gọi API bên ngoài [5]. Điều này giúp AI không chỉ tạo văn bản, mà có thể tương tác với các thành phần phần mềm khác.

Bài toán tư vấn sản phẩm trong AURA phù hợp với mô hình AI Agent vì đây là một nhiệm vụ nhiều bước. Khi người dùng đặt câu hỏi, hệ thống cần xử lý các bước sau:

1. Nhận và chuẩn hóa câu hỏi tự nhiên của người dùng.
2. Xác định ý định hội thoại, ví dụ tìm kiếm, so sánh, hỏi chi tiết, hỏi sản phẩm rẻ hơn, chọn sản phẩm đã hiển thị.
3. Trích xuất các ràng buộc như danh mục, thương hiệu, khoảng giá, mục đích sử dụng.
4. Kiểm tra trạng thái hội thoại trước đó để hiểu các cụm như “mẫu đó”, “hai mẫu này”, “cái vừa tư vấn”.
5. Truy xuất sản phẩm phù hợp từ catalog.
6. Kiểm tra dữ liệu và evidence.
7. Tạo câu trả lời có cấu trúc.
8. Cập nhật conversation_state cho lượt hội thoại tiếp theo.

Nếu toàn bộ các bước này được gộp vào một prompt duy nhất, hệ thống sẽ khó kiểm soát và khó kiểm thử. Vì vậy, AURA thiết kế AI advisor theo hướng workflow có các thành phần rõ ràng: intent router, planner, QueryFrame builder, product resolver, retrieval tool, response composer, verifier và state manager.

Việc tách agent thành nhiều bước có ba lợi ích chính. Thứ nhất, từng bước có thể được kiểm thử độc lập. Thứ hai, khi hệ thống trả lời sai, người phát triển có thể truy vết lỗi nằm ở intent, retrieval, state hay verifier. Thứ ba, hệ thống dễ mở rộng thêm kỹ năng tư vấn mới mà không phá vỡ toàn bộ pipeline.

Như vậy, AI Agent trong AURA không được hiểu là một mô hình tự trị hoàn toàn, mà là một agent workflow được giới hạn trong miền tư vấn sản phẩm công nghệ. Agent này chỉ được phép sử dụng các công cụ và dữ liệu do hệ thống cung cấp, từ đó giảm rủi ro trả lời ngoài phạm vi.

2.7. Harness Engineering cho AI Advisor

Trong phạm vi đề án này, harness engineering được hiểu là phương pháp thiết kế lớp điều phối và kiểm soát bao quanh AI Advisor. Lớp harness không thay thế mô hình AI, mà đóng vai trò quản lý toàn bộ quá trình agent xử lý yêu cầu: từ input, state, planning, retrieval, evidence, verification, recovery đến response contract và trace [18].

Cần nhấn mạnh rằng harness engineering trong đề án là một định nghĩa vận hành phục vụ thiết kế hệ thống. Đây chưa phải là một thuật ngữ được chuẩn hóa rộng rãi như RAG hoặc Transformer. Vì vậy, khi trình bày trong báo cáo, cần diễn đạt rõ: “Trong phạm vi đề tài, harness engineering được hiểu là...”. Cách viết này giúp báo cáo có

tính học thuật và tránh biến một thuật ngữ kỹ thuật thực hành thành khẳng định thiếu nguồn.

Harness cần thiết vì lỗi của AI Advisor thường không chỉ nằm ở câu trả lời cuối cùng. Một câu trả lời sai có thể xuất phát từ nhiều nguyên nhân khác nhau: hiểu sai ý định, kế thừa sai ngữ cảnh, chọn sai sản phẩm từ catalog, thiếu evidence, sinh claim quá mạnh, verifier không phát hiện lỗi hoặc frontend không render đúng response. Nếu chỉ kiểm tra câu trả lời cuối cùng, hệ thống sẽ khó biết lỗi nằm ở đâu.

Trong AURA, harness được thiết kế để kiểm soát các lớp sau:

- **Input Understanding:** tiếp nhận câu hỏi, chuẩn hóa văn bản, xác định intent.
- **AgentState:** lưu trạng thái hội thoại, sản phẩm đang được tập trung, danh sách sản phẩm đã hiển thị, ngân sách và mục đích sử dụng.
- **QueryFrame:** biểu diễn các ràng buộc truy vấn dưới dạng có cấu trúc.
- **Product Resolver:** xử lý tham chiếu như “mẫu đó”, “hai mẫu này”, mã sản phẩm hoặc tên model.
- **Retrieval/Tools:** tìm kiếm sản phẩm và lấy thông tin từ catalog.
- **Evidence Ledger:** ghi nhận các dữ kiện được phép dùng trong câu trả lời.
- **Verifier:** kiểm tra tính hợp lệ của câu trả lời.
- **Response Contract:** quy định cấu trúc dữ liệu trả về frontend.
- **Trace:** ghi lại các pha xử lý để phục vụ debug và evaluation.
- **Recovery:** đưa ra phản hồi an toàn khi preflight hoặc postflight phát hiện lỗi.

Nhờ harness, AI Advisor trong AURA không hoạt động như một chatbot tự do. Nó được đặt trong một hệ thống phần mềm có ràng buộc, có dữ liệu đầu vào rõ ràng, có bước kiểm chứng và có trace. Đây là một trong những đóng góp kỹ thuật chính của đề tài.

Ở góc độ thiết kế hệ thống, harness giúp biến AI Advisor từ một thành phần khó kiểm soát thành một workflow có thể phân tích, cài đặt, kiểm thử và đánh giá. Điều này đặc biệt quan trọng trong bối cảnh đưa AI vào hệ thống nghiệp vụ, nơi tính đúng đắn và khả năng truy vết quan trọng không kém chất lượng câu chữ.

2.8. Guardrails, verifier và kiểm chứng câu trả lời

Guardrails là các cơ chế kiểm tra, giới hạn hoặc can thiệp vào quá trình xử lý của hệ thống AI nhằm giảm rủi ro đầu vào và đầu ra. Trong các hệ thống agent, guardrails có thể được áp dụng trước khi agent thực hiện hành động, sau khi agent sinh kết quả hoặc trước khi gọi một công cụ cụ thể. Tài liệu OpenAI Agents SDK mô tả guardrails như một cơ chế hỗ trợ kiểm tra input, output và tool usage trong workflow agent [6].

Trong AURA, guardrails được chia thành hai nhóm chính: preflight và postflight. Preflight guardrails kiểm tra trước khi hệ thống thực hiện retrieval hoặc sinh câu trả lời. Mục tiêu của preflight là phát hiện các yêu cầu không hợp lệ, thiếu thông tin quan trọng hoặc có nguy cơ dẫn đến lỗi. Ví dụ, nếu người dùng hỏi “so sánh hai mẫu này”

nhưng trạng thái hội thoại không có hai sản phẩm trước đó, hệ thống cần yêu cầu người dùng chỉ rõ sản phẩm thay vì tự đoán.

Postflight guardrails kiểm tra sau khi hệ thống đã tạo câu trả lời. Mục tiêu của postflight là đảm bảo câu trả lời không vi phạm dữ liệu catalog và có thể hiển thị đúng trên frontend. Các lỗi cần kiểm tra bao gồm:

- Sản phẩm được nhắc trong câu trả lời không tồn tại trong catalog.
- Câu trả lời đề cập đến thông số không có trong evidence.
- Câu trả lời nói một sản phẩm phù hợp điều kiện nhưng sản phẩm không thỏa mãn QueryFrame.
- Danh sách related_products không khớp với nội dung answer_text.
- Câu trả lời không công bố missing_fields khi dữ liệu bị thiếu.
- Response không đúng contract nên frontend không thể render chính xác.

Verifier là thành phần thực hiện các kiểm tra này. Trong hệ thống AURA, verifier không chỉ kiểm tra văn bản, mà kiểm tra mối quan hệ giữa answer_text, related_products, evidence, missing_fields, verification metadata và conversation_state. Đây là điểm quan trọng vì câu trả lời của AI Advisor không tồn tại độc lập; nó phải đồng bộ với product cards và trạng thái hội thoại trên giao diện.

Guardrails và verifier giúp hệ thống chuyển từ cơ chế “AI trả lời và hy vọng đúng” sang cơ chế “AI trả lời trong giới hạn được kiểm chứng”. Nếu verifier phát hiện lỗi nghiêm trọng, hệ thống có thể chuyển sang safe_degrade, yêu cầu làm rõ hoặc trả lời rằng thông tin chưa đủ căn cứ. Cách phản hồi này có thể kém hấp dẫn hơn về mặt ngôn ngữ, nhưng đáng tin cậy hơn trong bối cảnh tư vấn sản phẩm.

Do đó, ở Chương 3 và Chương 4, phần thiết kế và kiểm thử cần trình bày rõ các lỗi mà verifier xử lý, ví dụ missing_field, unsupported_claim, product_not_found, constraint_mismatch và answer_cards_mismatch. Những lỗi này là bằng chứng cho thấy AI Advisor được kiểm soát bằng kỹ thuật phần mềm, không chỉ dựa vào prompt.

2.9. Evaluation cho AI Agent

Đánh giá một AI Agent không thể chỉ dừng ở việc đọc câu trả lời và nhận xét “có vẻ đúng”. Với một hệ thống nhiều bước như AURA, cần đánh giá cả quá trình xử lý, bao gồm intent, state, retrieval, evidence, verification, response contract và trace. Tài liệu về evaluation cho agent workflows nhấn mạnh vai trò của datasets, graders, traces và evaluation runs trong việc đánh giá hành vi của agent [7].

Trong hệ thống AURA, evaluation cần được nhìn theo nhiều lớp:

- **Unit test:** kiểm thử các hàm riêng lẻ như intent router, query parser, product resolver, verifier.
- **Contract test:** kiểm tra response từ API có đúng schema mà frontend yêu cầu hay không.

- **Scenario test:** kiểm thử các kịch bản hội thoại thực tế, ví dụ tìm sản phẩm theo ngân sách, hỏi tiếp “mẫu đó”, yêu cầu sản phẩm rẻ hơn, so sánh hai sản phẩm.
- **Trace-level evaluation:** đánh giá chuỗi phase của harness, bao gồm perceive, plan, retrieve, verify, recover và commit.
- **Regression test:** đảm bảo một lỗi đã sửa không xuất hiện lại khi thay đổi code.
- **Manual evaluation:** đọc câu trả lời, đối chiếu product cards và đánh giá tính hữu ích với người dùng.

Điểm cần chú ý là không nên trộn lẫn số liệu đã kiểm thử với số liệu kỳ vọng. Nếu báo cáo ghi “hệ thống đạt độ chính xác cao” nhưng không có dataset, metric hoặc log kiểm thử, nhận định đó sẽ yếu về mặt học thuật. Do đó, báo cáo cần tách rõ ba nhóm: kết quả đã chạy được, kết quả quan sát thủ công và kế hoạch đánh giá đề xuất.

Đối với AURA, các tiêu chí đánh giá phù hợp bao gồm:

- Tỷ lệ intent được phân loại đúng.
- Tỷ lệ QueryFrame trích xuất đúng ràng buộc.
- Tỷ lệ sản phẩm trả về thỏa mãn yêu cầu người dùng.
- Tỷ lệ câu trả lời có related_products khớp với nội dung text.
- Tỷ lệ verifier phát hiện claim không có evidence.
- Tỷ lệ hội thoại follow-up giữ đúng ngữ cảnh.
- Thời gian phản hồi trung bình của API chat.
- Tỷ lệ fallback hoặc safe_degrade trong các tình huống thiếu dữ liệu.

Nhờ cách đánh giá nhiều lớp, hệ thống có thể chứng minh không chỉ rằng AI Advisor “trả lời được”, mà còn rằng quá trình trả lời có thể kiểm soát, kiểm thử và cải thiện theo thời gian. Đây là yêu cầu quan trọng để một hệ thống AI trong đồ án tốt nghiệp có chiều sâu kỹ thuật thay vì chỉ dừng ở mức demo.

2.10. Công nghệ sử dụng trong dự án

Hệ thống AURA được xây dựng theo kiến trúc tách biệt frontend và backend. Frontend chịu trách nhiệm hiển thị giao diện người dùng, product cards và khung hội thoại AI Advisor. Backend chịu trách nhiệm cung cấp API, xử lý catalog, điều phối AI Advisor, kiểm chứng response và trả dữ liệu có cấu trúc cho frontend.

Việc lựa chọn công nghệ trong đồ án dựa trên các tiêu chí: dễ triển khai trong phạm vi prototype, hỗ trợ tách lớp rõ ràng, có khả năng kiểm thử, phù hợp với API contract và có thể mở rộng trong tương lai.

2.10.1. React, Vite và TypeScript

React là thư viện JavaScript phổ biến dùng để xây dựng giao diện người dùng theo hướng component-based [11]. Với React, giao diện có thể được chia thành các thành phần nhỏ như ProductGrid, ProductCard, CopilotDrawer, ChatMessage và các thành phần điều khiển trạng thái. Cách tổ chức này phù hợp với hệ thống AURA vì giao diện e-commerce và giao diện AI Advisor cần được cập nhật động theo dữ liệu từ API.

Vite là công cụ build frontend hiện đại, hỗ trợ khởi động nhanh môi trường phát triển và tối ưu quá trình build ứng dụng [12]. Trong đề án, Vite giúp quá trình phát triển frontend gọn nhẹ, phù hợp với mô hình prototype và dễ chạy trên localhost.

TypeScript được sử dụng để bổ sung kiểu dữ liệu tĩnh cho JavaScript [13]. Điều này đặc biệt hữu ích trong AURA vì frontend cần nhận các response phức tạp từ backend, bao gồm sản phẩm, `related_products`, `verification`, `conversation_state`, `active_context` và `decision_trace`. Khi các kiểu dữ liệu được định nghĩa rõ ràng, hệ thống giảm nguy cơ lỗi do sai tên trường hoặc sai cấu trúc dữ liệu.

Trong thiết kế AURA, React/Vite/TypeScript phục vụ các nhiệm vụ chính:

- Hiển thị danh sách sản phẩm và product cards.
- Hiển thị chi tiết sản phẩm và thông số nổi bật.
- Tạo giao diện CopilotDrawer cho AI Advisor.
- Gửi câu hỏi người dùng đến backend qua API.
- Nhận response contract và render `answer_text`, `related_products`, `warnings`, `missing_fields`.
- Lưu trạng thái hội thoại phía client khi cần.

Việc lựa chọn React và TypeScript phù hợp với yêu cầu đồng bộ dữ liệu giữa câu trả lời AI và giao diện sản phẩm. Nếu backend trả về sai contract, frontend có thể phát hiện sớm thông qua kiểu dữ liệu và logic render.

2.10.2. FastAPI, Pydantic và Uvicorn

FastAPI là framework Python hiện đại dùng để xây dựng API hiệu năng cao. FastAPI tận dụng type hints của Python để khai báo schema dữ liệu, validate input và tự động tạo tài liệu API theo chuẩn OpenAPI [8]. Điều này phù hợp với AURA vì hệ thống cần nhiều endpoint có response rõ ràng, đặc biệt là endpoint chat trả về cấu trúc phức tạp.

Pydantic là thư viện validation dữ liệu được tích hợp chặt chẽ với FastAPI. Trong AURA, các schema như `ChatRequest`, `ChatResponse`, `ChatTurn` hoặc các object liên quan đến `conversation_state` có thể được định nghĩa rõ ràng bằng Pydantic. Khi request không hợp lệ, hệ thống có thể phát hiện sớm trước khi đi vào pipeline AI Advisor [9].

Uvicorn là ASGI server dùng để chạy ứng dụng FastAPI. Với khả năng hỗ trợ async, Uvicorn phù hợp cho các API cần phản hồi nhanh và có thể xử lý nhiều request trong môi trường phát triển hoặc triển khai thử nghiệm [10].

Trong hệ thống AURA, FastAPI backend đảm nhận các chức năng chính:

- Cung cấp endpoint kiểm tra trạng thái hệ thống.
- Cung cấp API lấy danh sách sản phẩm.
- Cung cấp API lấy chi tiết sản phẩm theo mã sản phẩm.
- Cung cấp API lấy ảnh sản phẩm.

- Cung cấp API chat cho AI Advisor.
- Điều phối catalog service, AI service, harness runtime và verifier.
- Trả response contract về frontend.

Việc sử dụng FastAPI giúp hệ thống có ranh giới rõ giữa frontend và backend. Frontend không cần biết chi tiết AI Advisor xử lý ra sao, chỉ cần gọi API và render dữ liệu đúng contract. Backend chịu trách nhiệm kiểm soát logic nghiệp vụ và logic AI.

2.10.3. Python cho xử lý catalog và AI workflow

Python được lựa chọn cho backend và AI workflow vì hệ sinh thái Python có nhiều thư viện hỗ trợ xử lý dữ liệu, xây dựng API, kiểm thử và tích hợp AI. Trong AURA, Python không chỉ dùng để viết endpoint, mà còn dùng để xử lý catalog, phân tích query, ranking sản phẩm, xây dựng QueryFrame, kiểm chứng response và quản lý trace.

Các tác vụ phù hợp với Python trong hệ thống gồm:

- Đọc dữ liệu catalog từ CSV.
- Làm sạch và chuẩn hóa dữ liệu sản phẩm.
- Trích xuất giá trị số từ chuỗi giá.
- Phân tích câu hỏi tiếng Việt của người dùng.
- Phát hiện thương hiệu, danh mục, ngân sách, CPU, GPU và use_case.
- Tìm kiếm sản phẩm theo ràng buộc.
- Kiểm tra evidence và missing fields.
- Viết test cho các module AI Advisor.

Python phù hợp với đồ án vì vừa dễ triển khai prototype, vừa có khả năng mở rộng sang các workflow AI phức tạp hơn nếu hệ thống tiếp tục phát triển.

2.10.4. Catalog dạng CSV và hướng mở rộng sang cơ sở dữ liệu

Trong phiên bản hiện tại, catalog sản phẩm của AURA được tổ chức chủ yếu dưới dạng file CSV. Cách tiếp cận này phù hợp với giai đoạn prototype vì dễ chuẩn bị dữ liệu, dễ kiểm tra nội dung và không cần triển khai hệ quản trị cơ sở dữ liệu phức tạp. File catalog có thể chứa các trường như mã sản phẩm, tên sản phẩm, danh mục, thương hiệu, giá, mô tả, thông số, nguồn dữ liệu và provenance.

Tuy nhiên, CSV không phải là giải pháp tối ưu cho hệ thống production. Khi số lượng sản phẩm tăng, hoặc khi cần các chức năng như quản lý người dùng, đơn hàng, yêu thích, so sánh, lịch sử chat và quản trị catalog, hệ thống cần chuyển sang cơ sở dữ liệu quan hệ hoặc kết hợp với search engine.

Trong phạm vi thiết kế mở rộng, hệ thống có thể sử dụng cơ sở dữ liệu quan hệ với các bảng chính như Product, Brand, Category, ProductSpec, ProductImage, Conversation, Message, RecommendationTrace và EvaluationRun. Việc thiết kế lược đồ cơ sở dữ liệu sẽ được trình bày ở chương phân tích và thiết kế hệ thống.

Như vậy, CSV trong AURA được xem là nguồn dữ liệu catalog phục vụ prototype và kiểm chứng AI Advisor. Còn lược đồ cơ sở dữ liệu được xem là định hướng thiết kế để hệ thống có thể mở rộng thành sản phẩm hoàn chỉnh hơn.

2.10.5. Pytest cho kiểm thử

Pytest là framework kiểm thử phổ biến trong Python, hỗ trợ viết unit test, integration test và regression test một cách ngắn gọn [14]. Trong một hệ thống AI Advisor có nhiều thành phần như AURA, kiểm thử đóng vai trò quan trọng vì lỗi có thể xuất hiện ở nhiều lớp khác nhau.

Các nhóm kiểm thử phù hợp với AURA gồm:

- Kiểm thử intent router.
- Kiểm thử QueryFrame builder.
- Kiểm thử product resolver.
- Kiểm thử catalog search.
- Kiểm thử verifier.
- Kiểm thử API contract.
- Kiểm thử harness runtime.
- Kiểm thử các kịch bản hội thoại nhiều lượt.

Việc sử dụng pytest giúp báo cáo có bằng chứng kỹ thuật rõ ràng hơn. Thay vì chỉ mô tả hệ thống đã cài đặt, đồ án có thể chứng minh một số hành vi quan trọng đã được kiểm thử bằng test suite. Đây là điểm cần thiết để nâng chất lượng báo cáo từ mức mô tả repo lên mức đồ án có kiểm chứng.

2.10.6. Docker và Docker Compose

Docker là nền tảng container hóa giúp đóng gói ứng dụng cùng môi trường chạy của nó [15]. Docker Compose hỗ trợ định nghĩa và chạy nhiều service cùng lúc, ví dụ frontend và backend [16]. Với AURA, Docker Compose giúp chuẩn hóa quá trình triển khai local và giảm lỗi do khác biệt môi trường giữa các máy.

Trong đồ án, Docker có thể được sử dụng để:

- Đóng gói backend FastAPI.
- Đóng gói frontend React/Vite.
- Khởi động các service bằng một lệnh duy nhất.
- Tạo môi trường kiểm thử ổn định.
- Chuẩn bị nền tảng cho triển khai production trong tương lai.

Mặc dù hệ thống có thể chạy trực tiếp bằng npm và uvicorn trong môi trường phát triển, Docker Compose vẫn là lựa chọn quan trọng để thể hiện khả năng triển khai có hệ thống và dễ tái lập.

Kết luận chương

Chương này đã trình bày cơ sở lý thuyết và các công nghệ chính được sử dụng trong hệ thống AURA AI Sales Advisor. Về mặt lý thuyết, chương đã làm rõ vai trò của thương mại điện tử, tìm kiếm/lọc sản phẩm, đặc điểm dữ liệu sản phẩm công nghệ, LLM, RAG, AI Agent, harness engineering, guardrails, verifier và evaluation. Những nội dung này là nền tảng để hiểu vì sao AURA không được thiết kế như một chatbot tự do, mà là một hệ thống e-commerce tích hợp AI Advisor có kiểm soát.

Về mặt công nghệ, chương đã trình bày vai trò của React, Vite, TypeScript, FastAPI, Pydantic, Uvicorn, Python, CSV catalog, pytest và Docker Compose trong hệ thống. Các công nghệ này hỗ trợ việc tách lớp frontend/backend, định nghĩa API contract rõ ràng, xử lý catalog, xây dựng AI workflow và kiểm thử hành vi hệ thống.

Từ các cơ sở trên, chương tiếp theo sẽ trình bày quá trình phân tích và thiết kế hệ thống AURA, bao gồm yêu cầu hệ thống, tác nhân, use case, kịch bản use case, kiến trúc hệ thống, biểu đồ lớp và thiết kế dữ liệu.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

Chương này trình bày quá trình phân tích và thiết kế hệ thống AURA AI Sales Advisor. Cách trình bày được điều chỉnh theo tinh thần của tài liệu đồ án mẫu: trước hết mô tả hệ thống bằng ngôn ngữ tự nhiên, sau đó xác định yêu cầu, tác nhân, biểu đồ use case, kịch bản use case, thiết kế kiến trúc, biểu đồ lớp và lược đồ cơ sở dữ liệu [19]. Khác với bản trước, chương này hạn chế dùng bảng; các nội dung phân tích được chuyển sang dạng đoạn văn, gạch đầu dòng và mô tả kịch bản để báo cáo tự nhiên hơn, đỡ giống bản liệt kê.

Trong phạm vi đề tài, AURA không được xem như một chatbot độc lập. Hệ thống được thiết kế như một storefront thương mại điện tử cho sản phẩm công nghệ, trong đó AI Advisor được đặt bên cạnh catalog để hỗ trợ người dùng lựa chọn sản phẩm dựa trên dữ liệu thật. Vì vậy, phần thiết kế cần thể hiện đồng thời hai lớp: lớp thương mại điện tử dùng để hiển thị và tra cứu sản phẩm, và lớp AI harness dùng để kiểm soát câu trả lời tự vấn.

3.1. Mô tả hệ thống bằng ngôn ngữ tự nhiên

AURA AI Sales Advisor là hệ thống thương mại điện tử hỗ trợ tư vấn sản phẩm công nghệ. Khách hàng truy cập website để xem catalog gồm laptop và điện thoại. Mỗi sản phẩm được hiển thị dưới dạng product card, bao gồm tên sản phẩm, mã sản phẩm, thương hiệu, giá bán, ảnh và các thông số nổi bật. Người dùng có thể tìm kiếm hoặc lọc sản phẩm theo danh mục, thương hiệu, giá và một số thông số kỹ thuật quan trọng như RAM, SSD, CPU hoặc GPU.

Khi người dùng chưa biết nên lọc theo tiêu chí nào, họ có thể sử dụng AI Advisor để đặt câu hỏi bằng tiếng Việt tự nhiên. Ví dụ, người dùng có thể hỏi “cho tôi laptop Dell

dưới 20 triệu”, “mẫu đó RAM bao nhiêu”, “so sánh hai mẫu này”, hoặc “có máy nào hợp lập trình hơn không”. Hệ thống sẽ phân tích câu hỏi, tìm sản phẩm phù hợp trong catalog và trả lời kèm product cards liên quan để người dùng đối chiếu.

Điểm quan trọng của AURA là AI Advisor không được để trả lời tự do hoàn toàn. Mỗi lượt hỏi được đưa qua một lớp harness. Lớp này thực hiện các bước: phân loại ý định, tạo QueryFrame, quản lý trạng thái hội thoại, resolve tham chiếu sản phẩm, truy xuất catalog, ghi evidence, kiểm chứng bằng verifier và trả response contract cho frontend. Nhờ đó, câu trả lời của AI vừa có khả năng hiểu ngôn ngữ tự nhiên, vừa bị ràng buộc bởi dữ liệu catalog thật.

Bên cạnh chức năng dành cho khách hàng, hệ thống còn có nhóm chức năng kỹ thuật phục vụ nạp dữ liệu, làm sạch dữ liệu, kiểm tra catalog, theo dõi health, metrics, trace và chạy bộ kiểm thử. Các chức năng này giúp chứng minh hệ thống không chỉ là giao diện demo mà có quy trình vận hành và kiểm chứng tương đối rõ ràng.

3.2. Phân tích hệ thống

3.2.1. Yêu cầu hệ thống

Từ mô tả hệ thống, các yêu cầu được chia thành hai nhóm: yêu cầu chức năng và yêu cầu phi chức năng. Trong bản này, thay vì đưa toàn bộ yêu cầu vào bảng, các yêu cầu được trình bày theo dạng mô tả để người đọc thấy rõ vai trò của từng chức năng trong hệ thống.

a) Yêu cầu chức năng

Nhóm yêu cầu chức năng của AURA tập trung vào ba mảng chính: catalog sản phẩm, AI Advisor và vận hành kỹ thuật.

- F01 - Xem catalog sản phẩm: khách hàng có thể xem danh sách sản phẩm công nghệ trên giao diện chính. Danh sách hiển thị các thông tin cơ bản như tên, mã sản phẩm, thương hiệu, giá, ảnh và thông số nổi bật.
- F02 - Tìm kiếm và lọc sản phẩm: hệ thống cho phép người dùng tìm theo từ khóa và lọc theo danh mục, thương hiệu, mức giá hoặc thông số kỹ thuật. Đây là chức năng nền để người dùng tự khám phá sản phẩm trước khi hỏi AI.
- F03 - Xem chi tiết sản phẩm: khi chọn một product card, người dùng có thể xem thông tin chi tiết của sản phẩm, bao gồm mô tả, thông số, ảnh và nguồn dữ liệu nếu có.
- F04 - Hỏi AI Advisor: người dùng có thể nhập câu hỏi tiếng Việt tự nhiên vào khung tư vấn. Hệ thống gửi câu hỏi tới API chat và nhận về câu trả lời có cấu trúc.

- F05 - Hỏi tiếp theo ngữ cảnh: hệ thống cần hiểu các cụm như “mẫu đó”, “con thứ hai”, “mẫu vừa tư vấn” dựa trên trạng thái hội thoại trước đó.
- F06 - So sánh và chọn sản phẩm: người dùng có thể yêu cầu so sánh hai hoặc nhiều sản phẩm, hoặc hỏi nên chọn sản phẩm nào theo một mục đích sử dụng cụ thể.
- F07 - Đồng bộ câu trả lời với product cards: sản phẩm được nhắc trong câu trả lời phải khớp với các product cards được hiển thị kèm theo.
- F08 - Xử lý thiếu dữ liệu: nếu catalog thiếu một thông số như pin, trọng lượng hoặc độ bền, hệ thống cần công bố thiếu dữ liệu thay vì tự suy đoán.
- F09 - Theo dõi health, metrics và trace: developer/tester có thể kiểm tra trạng thái backend, trạng thái catalog, trạng thái AI workflow và trace của từng lượt xử lý.
- F10 - Quản lý catalog pipeline: hệ thống cần có quy trình thu thập, làm sạch, chuẩn hóa, kiểm tra và cập nhật dữ liệu sản phẩm.
- F11 - Kiểm thử và evaluation: các thành phần như intent routing, product resolver, verifier, response contract và harness cần có test tương ứng.
- F12 - Cấu hình AI workflow: hệ thống có thể chạy ở chế độ deterministic catalog mode hoặc bật external AI khi có cấu hình phù hợp.

b) Yêu cầu phi chức năng

Ngoài các chức năng cụ thể, AURA cần đáp ứng một số yêu cầu phi chức năng để phù hợp với bài toán đưa AI vào hệ thống nghiệp vụ.

- Tính đúng đắn dữ liệu: mọi thông tin tư vấn về giá, cấu hình và mã sản phẩm phải dựa trên catalog, không dùng trí nhớ của mô hình ngôn ngữ làm nguồn sự thật.
- Tính kiểm chứng: mỗi câu trả lời quan trọng cần có thể truy vết qua evidence, verification và trace.
- Tính nhất quán giao diện: nội dung văn bản của AI và product cards phải cùng tham chiếu đến một tập sản phẩm.
- Tính ổn định hội thoại: trạng thái hội thoại cần đủ để xử lý follow-up, nhưng không được giữ ngữ cảnh sai quá lâu khi người dùng đổi chủ đề.
- Tính bảo trì: các phần frontend, API, catalog service, AI harness, verifier và tests cần tách thành module rõ ràng.
- Tính mở rộng: hệ thống hiện có thể dùng CSV catalog trong prototype, nhưng thiết kế cần cho phép chuyển sang cơ sở dữ liệu quan hệ khi phát triển production.

- Tính quan sát: hệ thống cần có health check, metrics và trace để đánh giá agent nhiều bước.
- An toàn cấu hình: các khóa API, biến môi trường và thông tin nhạy cảm không được commit trực tiếp vào mã nguồn.

3.2.2. Xác định tác nhân

Tác nhân của hệ thống được xác định theo vai trò tương tác với hệ thống. AI Advisor là module nằm bên trong hệ thống nên không được xem là actor độc lập. Các actor chính gồm khách hàng, quản trị catalog, developer/tester, nguồn dữ liệu sản phẩm và LLM provider tùy chọn.

Khách hàng là actor chính của hệ thống. Khách hàng truy cập website để xem catalog, tìm kiếm sản phẩm, lọc sản phẩm, mở chi tiết sản phẩm, đặt câu hỏi cho AI Advisor, hỏi tiếp theo ngữ cảnh và xem câu trả lời kèm các product cards liên quan.

Quản trị catalog là actor vận hành dữ liệu. Trong prototype, vai trò này có thể do nhóm phát triển đảm nhiệm thay vì có một giao diện quản trị đầy đủ. Nhiệm vụ chính của quản trị catalog là chuẩn bị dữ liệu, kiểm tra dữ liệu, cập nhật file catalog, quản lý ảnh sản phẩm và đảm bảo các thông số quan trọng được chuẩn hóa.

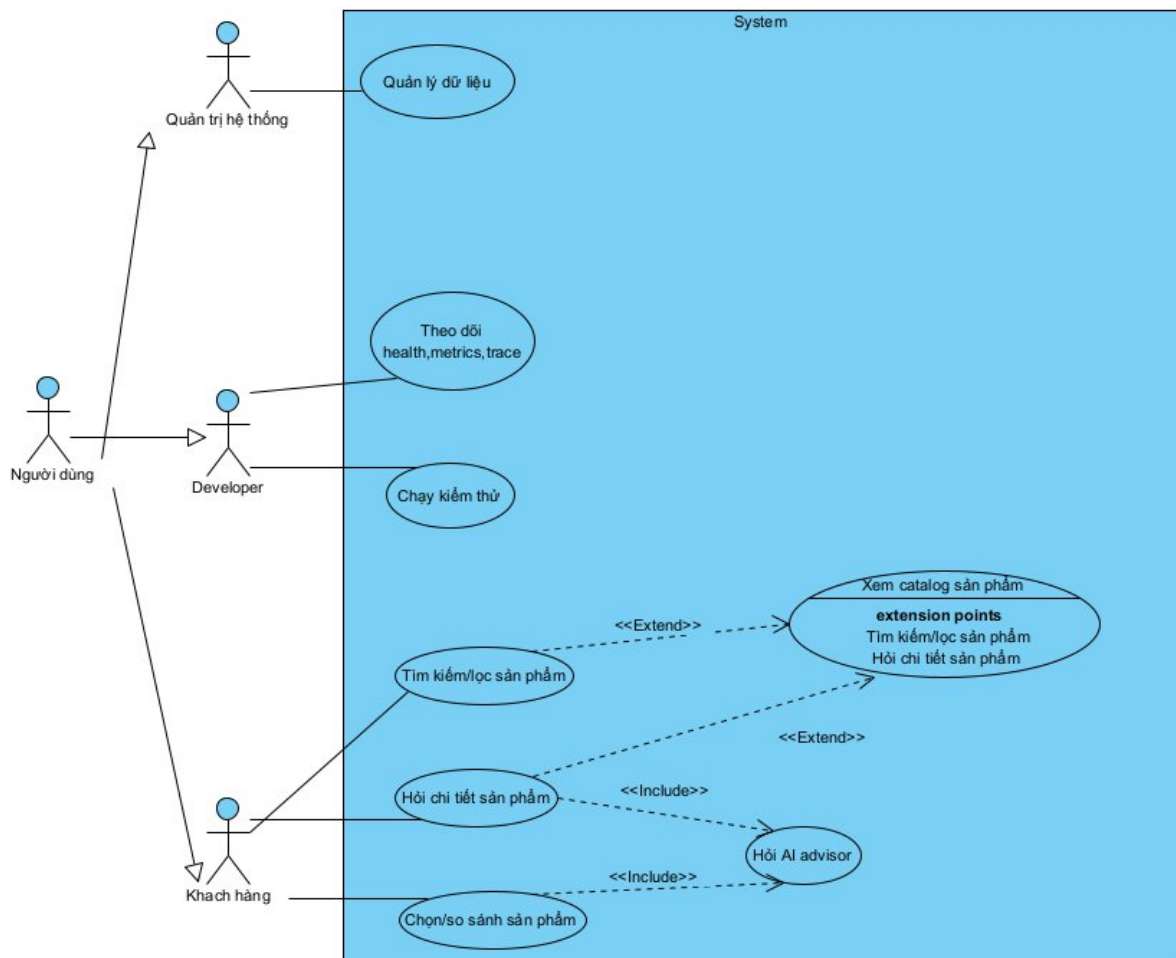
Developer/Tester là actor kỹ thuật. Vai trò này chạy hệ thống, cấu hình môi trường, gọi endpoint health/metrics, xem decision trace và chạy các bộ kiểm thử để đánh giá hành vi của harness.

Nguồn dữ liệu sản phẩm là actor bên ngoài, cung cấp dữ liệu thô về tên sản phẩm, giá, ảnh, thông số, URL nguồn và thời điểm cập nhật. Nguồn này không thao tác trực tiếp trên giao diện nhưng ảnh hưởng đến chất lượng tư vấn.

LLM provider là hệ thống ngoài tùy chọn. Khi external AI workflow được bật, LLM provider có thể tham gia vào việc diễn đạt câu trả lời hoặc hỗ trợ một số bước suy luận. Tuy nhiên, dữ liệu sản phẩm vẫn phải lấy từ catalog của AURA.

3.2.3. Biểu đồ Use case tổng quát

Biểu đồ use case tổng quát thể hiện các nhóm tương tác chính giữa actor và hệ thống. Khách hàng tương tác với các chức năng xem catalog, tìm kiếm/lọc, xem chi tiết, hỏi AI Advisor, hỏi tiếp theo ngữ cảnh và nhận câu trả lời kèm product cards. Quản trị catalog tương tác với chức năng quản lý dữ liệu catalog. Developer/Tester tương tác với nhóm chức năng health, metrics, trace và kiểm thử.



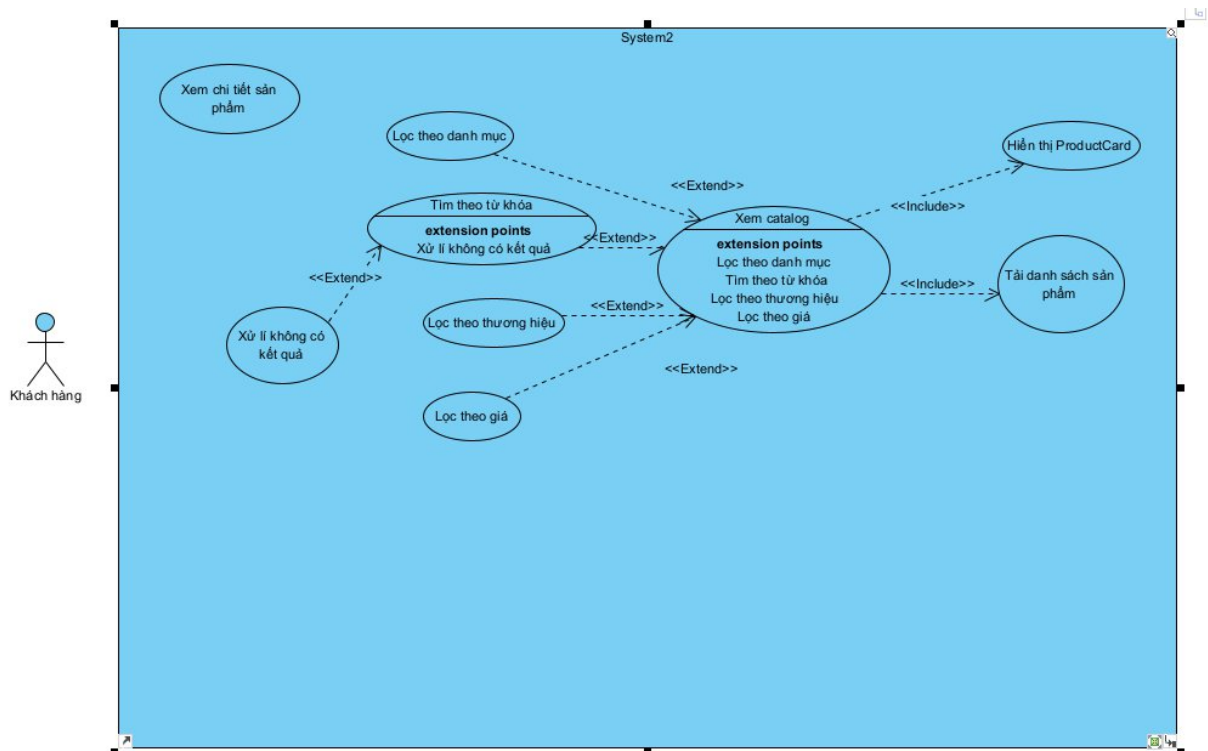
Hình 3.1. Biểu đồ use case tổng quát của hệ thống AURA

Trong biểu đồ tổng quát, use case “Hỏi AI Advisor” không đứng tách biệt khỏi storefront mà được gắn với “Nhận câu trả lời + product cards”. Điều này thể hiện luận điểm thiết kế: AI Advisor phải trả lời trong mỗi liên hệ trực tiếp với catalog và giao diện sản phẩm.

3.2.4. Biểu đồ Use case chi tiết và kịch bản

a) Nhóm use case của Khách hàng

Nhóm use case của khách hàng tập trung vào trải nghiệm mua sắm và tư vấn. Người dùng có thể xem catalog, tìm kiếm/lọc sản phẩm, xem chi tiết sản phẩm, hỏi AI Advisor, hỏi tiếp theo ngữ cảnh, so sánh sản phẩm và nhận câu trả lời kèm product cards.



Hình 3.2. Biểu đồ use case chi tiết nhóm chức năng Khách hàng

UC01 - Xem catalog sản phẩm

Tác nhân chính: Khách hàng

Tiền điều kiện: Frontend và backend đang chạy; catalog đã được nạp thành công.

Chuỗi sự kiện chính:

1. Người dùng mở trang chính của hệ thống.
2. Frontend gọi API danh sách sản phẩm.
3. Backend sử dụng CatalogService để lấy dữ liệu từ catalog.
4. Frontend hiển thị ProductGrid và ProductCard với tên, mã, thương hiệu, giá, ảnh và thông số nổi bật.
5. Người dùng có thể chọn sản phẩm để xem chi tiết hoặc tiếp tục hỏi AI Advisor.

Ngoại lệ:

- Catalog rỗng hoặc backend lỗi thì hệ thống hiển thị thông báo không tải được sản phẩm.
- Sản phẩm thiếu ảnh thì hệ thống sử dụng ảnh mapped, ảnh generic hoặc placeholder phù hợp.

Hậu điều kiện: Danh sách sản phẩm được hiển thị và trở thành điểm neo trực quan cho các thao tác tiếp theo.

UC02 - Tìm kiếm và lọc sản phẩm

Tác nhân chính: Khách hàng

Tiền điều kiện: Catalog đã sẵn sàng và người dùng đang ở trang sản phẩm.

Chuỗi sự kiện chính:

1. Người dùng nhập từ khóa hoặc chọn bộ lọc theo danh mục/thương hiệu/mức giá.
2. Frontend gửi query và filter đến API sản phẩm.
3. CatalogService chuẩn hóa query, phân tích price intent nếu có và lọc sản phẩm theo điều kiện.
4. Backend trả danh sách sản phẩm phù hợp.
5. ProductGrid cập nhật kết quả để người dùng xem hoặc chọn sản phẩm.

Ngoại lệ:

- Không có sản phẩm phù hợp thì hệ thống hiển thị thông báo không tìm thấy kết quả.
- Filter quá hẹp thì người dùng có thể bỏ bớt điều kiện hoặc chuyển sang hỏi AI Advisor.

Hậu điều kiện: Tập sản phẩm ứng viên được thu hẹp theo nhu cầu người dùng.

UC03 - Xem chi tiết sản phẩm

Tác nhân chính: Khách hàng

Tiền điều kiện: Người dùng đang xem ProductCard hoặc danh sách kết quả.

Chuỗi sự kiện chính:

1. Người dùng chọn nút xem chi tiết trên product card.
2. Frontend lấy product code của sản phẩm.
3. Backend truy xuất chi tiết sản phẩm theo product code.
4. Nếu sản phẩm tồn tại, hệ thống trả về tên, giá, thương hiệu, specs, ảnh và nguồn dữ liệu nếu có.
5. Frontend mở modal hoặc drawer chi tiết sản phẩm.

Ngoại lệ:

- Product code không tồn tại thì backend trả lỗi không tìm thấy sản phẩm.
- Nếu ảnh cục bộ không có, hệ thống dùng ảnh mapped hoặc ảnh generic.

Hậu điều kiện: Sản phẩm đang xem có thể được dùng làm ngữ cảnh cho câu hỏi tiếp theo.

UC04 - Hỏi AI Advisor

Tác nhân chính: Khách hàng

Tiền điều kiện: CopilotDrawer đã mở; backend chat API sẵn sàng.

Chuỗi sự kiện chính:

1. Người dùng nhập câu hỏi tiếng Việt tự nhiên.
2. Frontend gửi message, history và conversation_state tới API chat.
3. Backend khởi tạo trace cho lượt hỏi.
4. Harness phân tích intent, tạo QueryFrame và xác định có cần kế thừa ngữ cảnh cũ hay không.
5. CatalogService truy xuất hoặc resolve sản phẩm liên quan.
6. AIService hoặc deterministic advisor tạo câu trả lời nháp dựa trên candidates và evidence.
7. Verifier kiểm tra câu trả lời, product codes, missing fields và text-card consistency.
8. Backend trả ChatResponse cho frontend.
9. Frontend hiển thị câu trả lời và các product cards liên quan.

Ngoại lệ:

- Nếu câu hỏi thiếu thông tin, hệ thống hỏi lại thay vì đoán.
- Nếu postflight verification thất bại, hệ thống chuyển sang safe-degrade hoặc cảnh báo.

Hậu điều kiện: Conversation state được cập nhật để phục vụ câu hỏi tiếp theo.

UC05 - Hỏi tiếp theo ngữ cảnh

Tác nhân chính: Khách hàng

Tiền điều kiện: Đã có ít nhất một lượt AI Advisor trả về candidate hoặc active product.

Chuỗi sự kiện chính:

1. Người dùng hỏi bằng tham chiếu như “mẫu đó”, “con thứ hai”, “mẫu vừa tư vấn”.
2. Backend đọc conversation_state và danh sách candidate đã hiển thị.
3. ProductResolver xác định sản phẩm được nhắc tới theo active_product_code, ordinal hoặc tên model.
4. QueryFrame kế thừa category, budget hoặc use case nếu vẫn cùng chủ đề.
5. Hệ thống trả lời tập trung vào đúng sản phẩm đã resolve.

Ngoại lệ:

- Nếu tham chiếu mơ hồ, hệ thống yêu cầu người dùng chỉ rõ sản phẩm.
- Nếu sản phẩm trong state đã không còn trong catalog, hệ thống thông báo không thể xác minh.

Hậu điều kiện: Focused product hoặc active product được cập nhật trong state.

UC06 - So sánh sản phẩm

Tác nhân chính: Khách hàng

Tiền điều kiện: Người dùng đã chọn hoặc nhắc đến ít nhất hai sản phẩm/thương hiệu.

Chuỗi sự kiện chính:

1. Người dùng yêu cầu so sánh hai mẫu hoặc hỏi nên chọn sản phẩm nào.
2. Resolver xác định các sản phẩm cần so sánh.
3. Preflight kiểm tra các sản phẩm có cùng nhóm so sánh hợp lý hay không.
4. Hệ thống lấy facts quan trọng như giá, CPU, RAM, SSD, GPU và màn hình.
5. Response composer tạo câu trả lời so sánh ngắn gọn, nêu điểm mạnh/yếu theo mục đích sử dụng.
6. Verifier kiểm tra các claim mạnh và sự nhất quán giữa text và product cards.

Ngoại lệ:

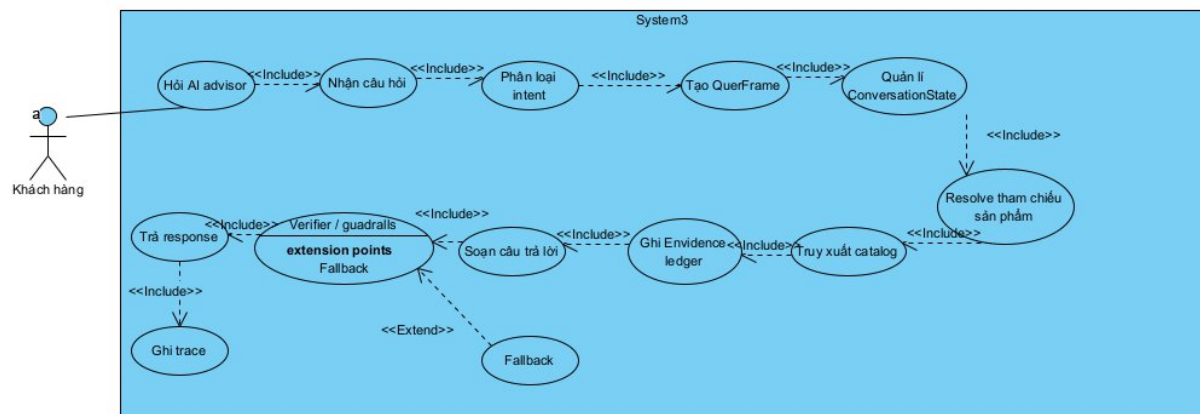
- Nếu sản phẩm khác loại hoặc thiếu tiêu chí so sánh, hệ thống hỏi lại người dùng.

- Nếu thiếu evidence cho một tiêu chí, hệ thống chỉ nói “catalog chưa có dữ liệu” thay vì kết luận.

Hậu điều kiện: Compared codes và compared brands được cập nhật trong state.

b) Nhóm use case của AI Advisor và harness

Nhóm use case này không mô tả AI Advisor như một actor bên ngoài, mà mô tả các chức năng nội bộ mà hệ thống phải thực hiện để tạo câu trả lời có kiểm soát. Các chức năng này gồm phân tích intent, tạo QueryFrame, resolve sản phẩm, truy xuất catalog, ghi evidence, kiểm chứng và trả response contract.



Hình 3.3. Biểu đồ use case chi tiết nhóm chức năng AI Advisor và harness

UC07 - Tạo QueryFrame từ câu hỏi tự nhiên

Tác nhân chính: Khách hàng, thông qua AI Advisor

Tiền điều kiện: Người dùng gửi câu hỏi có nội dung tự vấn hoặc tìm kiếm.

Chuỗi sự kiện chính:

1. Hệ thống chuẩn hóa câu hỏi đầu vào.
2. Planner xác định dialogue act như tìm kiếm, hỏi chi tiết, so sánh, reject candidate hoặc refine preference.
3. QueryFrame builder rút ra category, brand, budget, CPU/GPU filters, use_case và requested_attributes.
4. Nếu câu hỏi là follow-up, QueryFrame có thể kế thừa thông tin từ DecisionContext.
5. QueryFrame được dùng làm đầu vào cho retrieval và verifier.

Ngoại lệ:

- Nếu câu hỏi quá mơ hồ, hệ thống trả về yêu cầu làm rõ.
- Nếu người dùng đổi chủ đề rõ ràng, hệ thống tạo topic mới thay vì kế thừa state cũ.

Hậu điều kiện: Yêu cầu tự nhiên được biểu diễn thành cấu trúc có thể kiểm thử.

UC08 - Kiểm chứng câu trả lời

Tác nhân chính: Hệ thống

Tiền điều kiện: Đã có câu trả lời nháp, danh sách sản phẩm ứng viên và evidence.

Chuỗi sự kiện chính:

1. Verifier nhận answer, product codes, candidate set, evidence refs và QueryFrame.
2. Verifier kiểm tra sản phẩm được nhắc trong câu trả lời có tồn tại trong catalog không.
3. Verifier kiểm tra sản phẩm có thỏa ràng buộc category, brand và budget không.
4. Verifier phát hiện claim không có evidence hoặc trường dữ liệu bị thiếu nhưng không được công bố.
5. Verifier kiểm tra text và related product cards có cùng nói về một tập sản phẩm không.
6. Nếu kiểm chứng đạt, hệ thống commit state và trả response. Nếu không đạt, hệ thống safe-degrade hoặc hỏi lại.

Ngoại lệ:

- Nếu lỗi nghiêm trọng, câu trả lời nháp bị chặn.
- Nếu chỉ là warning, hệ thống có thể trả lời kèm cảnh báo hoặc missing_fields.

Hậu điều kiện: Câu trả lời cuối cùng giảm rủi ro hallucination và mismatch giao diện.

UC09 - Hiển thị answer đồng bộ với product cards

Tác nhân chính: Khách hàng

Tiền điều kiện: API chat trả về ChatResponse hợp lệ.

Chuỗi sự kiện chính:

1. Frontend đọc answer_text hoặc text trong ChatResponse.
2. Frontend đọc products hoặc related_products được backend gửi kèm.

3. ProductCard ưu tiên display_specs phù hợp với câu hỏi nếu có.
4. Inline product cards được render ngay trong vùng chat.
5. Người dùng đối chiếu câu trả lời AI với sản phẩm thật trên giao diện.

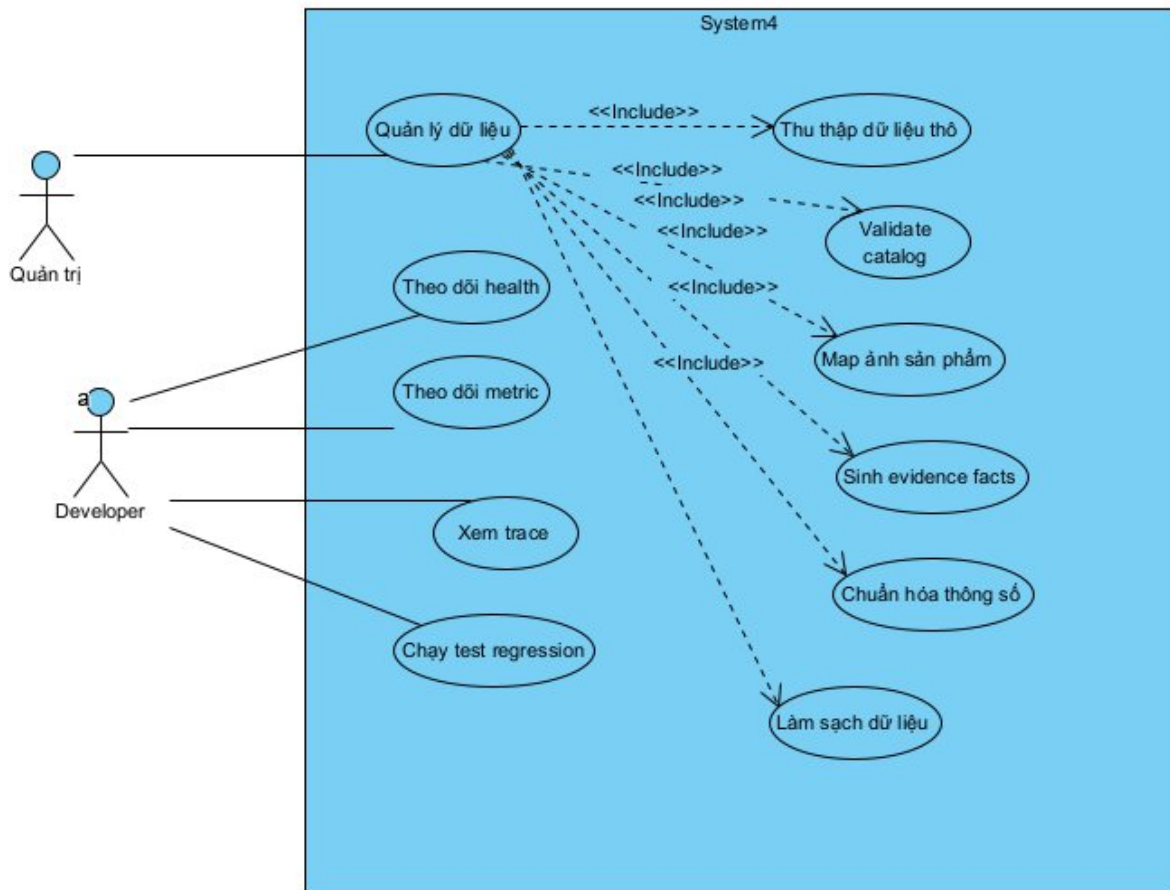
Ngoại lệ:

- Nếu related_products rỗng, hệ thống chỉ hiển thị text và cảnh báo phù hợp.
- Nếu contract lỗi, frontend hiển thị fallback thay vì render sai.

Hậu điều kiện: Người dùng có sản phẩm cụ thể để tiếp tục hỏi, so sánh hoặc xem chi tiết.

c) Nhóm use case quản trị catalog và vận hành kỹ thuật

Nhóm use case này phục vụ việc chuẩn bị dữ liệu và đánh giá kỹ thuật. Trong prototype, nhóm chức năng này có thể được thao tác bằng script, endpoint hoặc lệnh test thay vì giao diện quản trị đầy đủ.



Hình 3.4. Biểu đồ use case chi tiết nhóm chức năng quản trị catalog và vận hành kỹ thuật

UC10 - Nạp và chuẩn hóa catalog

Tác nhân chính: Quản trị catalog

Tiền điều kiện: Có nguồn dữ liệu sản phẩm thô.

Chuỗi sự kiện chính:

1. Thu thập dữ liệu sản phẩm từ nguồn bên ngoài hoặc file thô.
2. Làm sạch tên sản phẩm, thương hiệu, danh mục, giá và mã sản phẩm.
3. Chuẩn hóa thông số như CPU, RAM, SSD, GPU, màn hình, pin, trọng lượng nếu có.
4. Sinh các facts và provenance để phục vụ grounding.
5. Validate catalog và lưu thành catalog sạch cho hệ thống sử dụng.

Ngoại lệ:

- Nếu dữ liệu thiếu field quan trọng, hệ thống đánh dấu missing thay vì tự điền bù.

- Nếu giá không parse được, sản phẩm cần được sửa thủ công hoặc loại khỏi filter giá.

Hậu điều kiện: Catalog sạch sẵn sàng cho API và AI Advisor.

UC11 - Theo dõi health, metrics và trace

Tác nhân chính: Developer/Tester

Tiền điều kiện: Backend đang chạy.

Chuỗi sự kiện chính:

1. Developer gọi endpoint health để kiểm tra hệ thống sẵn sàng, số lượng sản phẩm và trạng thái AI.
2. Developer gọi endpoint metrics để xem thống kê agent/harness.
3. Khi cần debug, developer bật expose decision trace.
4. Trace được dùng để kiểm tra các pha xử lý như plan, retrieve, verify, recover và commit.

Ngoại lệ:

- Nếu external AI chưa cấu hình, hệ thống vẫn có thể chạy ở deterministic catalog mode.
- Nếu trace thiếu terminal event, cần kiểm tra lại harness runtime.

Hậu điều kiện: Có dữ liệu để debug và chứng minh workflow nhiều bước.

UC12 - Chạy kiểm thử và evaluation

Tác nhân chính: Developer/Tester

Tiền điều kiện: Môi trường test đã được cài đặt.

Chuỗi sự kiện chính:

1. Developer chạy test suite bằng pytest.
2. Các test kiểm tra API contract, product resolver, display specs, verifier và harness runtime.
3. Kết quả test được ghi lại để phục vụ Chương 4.
4. Các lỗi phát hiện được dùng để điều chỉnh thiết kế hoặc bổ sung guardrails.

Ngoại lệ:

- Nếu test thất bại, developer xem trace/log để xác định lỗi ở intent, state, retrieval, verifier hay response contract.
- Nếu test phụ thuộc external AI, hệ thống cần fallback hoặc mock để kết quả ổn định.

Hậu điều kiện: Có bằng chứng kỹ thuật cho các chức năng chính của hệ thống.

3.3. Thiết kế hệ thống

3.3.1. Kiến trúc tổng thể

Kiến trúc của AURA được tổ chức theo hướng tách frontend và backend. Frontend chịu trách nhiệm hiển thị catalog, product cards, chi tiết sản phẩm và khung hội thoại AI Advisor. Backend chịu trách nhiệm cung cấp API, đọc và lọc catalog, điều phối AI Advisor, kiểm chứng câu trả lời và trả dữ liệu có cấu trúc cho frontend.

Có thể chia kiến trúc thành bốn lớp chính. Lớp Presentation gồm ProductGrid, ProductCard, CopilotDrawer và các thành phần hiển thị chat. Lớp API gồm các endpoint như sản phẩm, ảnh, chat, health và metrics. Lớp Domain/Harness gồm CatalogService, Planner, QueryFrame, ProductResolver, EvidenceLedger, Verifier và TraceCollector. Lớp Data hiện tại dùng CSV catalog và file ảnh, đồng thời có thể mở rộng sang cơ sở dữ liệu quan hệ trong phiên bản production.

Cách tổ chức này giúp tách biệt giao diện người dùng khỏi logic tư vấn. Frontend không cần biết AI Advisor hoạt động chi tiết thế nào; frontend chỉ nhận response contract và render. Backend kiểm soát logic nghiệp vụ, dữ liệu và các ràng buộc kiểm chứng.

3.3.2. Thiết kế luồng xử lý catalog

Catalog là nguồn sự thật chính của hệ thống. Khi người dùng mở trang sản phẩm hoặc dùng bộ lọc, frontend gửi request tới backend. Backend gọi CatalogService để đọc catalog, chuẩn hóa query, xử lý điều kiện lọc và serialize sản phẩm thành dạng dữ liệu frontend có thể hiển thị.

Luồng catalog gồm các bước: nạp dữ liệu từ file catalog, tạo chỉ mục theo product code, chuẩn hóa giá để phục vụ lọc ngân sách, trích xuất thông số kỹ thuật để hiển thị, resolve ảnh sản phẩm và trả danh sách product dict cho frontend. Với các câu hỏi AI, CatalogService còn được dùng để truy xuất candidate phù hợp với QueryFrame.

Trong prototype, catalog được lưu dưới dạng CSV để dễ kiểm tra và cập nhật. Tuy nhiên, phần thiết kế dữ liệu vẫn giữ khả năng mở rộng sang database quan hệ. Khi số

lượng sản phẩm tăng hoặc cần lưu lịch sử hội thoại/trace lâu dài, việc chuyển sang database là cần thiết.

3.3.3. Thiết kế luồng xử lý AI Advisor

Luồng xử lý AI Advisor bắt đầu khi người dùng nhập câu hỏi vào CopilotDrawer. Frontend gửi message, history và conversation_state lên API chat. Backend khởi tạo một lượt chạy harness, sau đó lần lượt thực hiện các bước: phân tích intent, tạo QueryFrame, resolve sản phẩm, truy xuất catalog, tạo câu trả lời nháp, kiểm chứng và commit state.

Luồng thể hiện rõ AI Advisor không phải là một hàm sinh text đơn giản. Mỗi câu trả lời được tạo ra sau khi đã đi qua các bước kiểm soát. Nếu preflight phát hiện thiếu thông tin, hệ thống có thể hỏi lại. Nếu postflight hoặc verifier phát hiện lỗi, hệ thống có thể chặn câu trả lời nháp và chuyển sang safe-degrade.

Kết quả cuối cùng trả về frontend là ChatResponse. Response này gồm text, products hoặc related_products, verification, conversation_state, follow_up_question và decision_trace nếu chế độ debug được bật. Thiết kế response contract giúp frontend render ổn định và giúp hệ thống có thể test được.

3.4. Xây dựng biểu đồ lớp

3.4.1. Trích rút danh từ và lựa chọn danh từ làm thực thể

Dựa trên mô tả hệ thống, các danh từ quan trọng được trích rút gồm: khách hàng, catalog, sản phẩm, mã sản phẩm, thương hiệu, danh mục, giá, ảnh, thông số, câu hỏi, phiên hội thoại, trạng thái hội thoại, QueryFrame, evidence, verifier, trace, product card và AI Advisor. Không phải mọi danh từ đều trở thành lớp thực thể. Một số danh từ chỉ là thuộc tính, một số là actor, một số là lớp biên hoặc lớp điều khiển.

Nhóm danh từ được chọn làm lớp thực thể gồm Product, Category, Brand, ProductSpec, ProductImage, EvidenceFact, ConversationSession, ChatMessage, AdvisorRun, VerifierIssue và TraceEvent. Đây là những đối tượng có dữ liệu cần quản lý hoặc có thể lưu trữ trong cơ sở dữ liệu khi hệ thống mở rộng.

Nhóm danh từ được chọn làm lớp điều khiển gồm CatalogService, ConversationPlanner, QueryFrameBuilder, ProductResolver, AdvisorHarness, AIService, Verifier và TraceCollector. Các lớp này không nhất thiết là bảng dữ liệu, nhưng là thành phần quan trọng trong thiết kế phần mềm vì chúng thực hiện logic nghiệp vụ và logic kiểm chứng.

Nhóm danh từ được chọn làm lớp biên gồm ProductGrid, ProductCard, CopilotDrawer, FastAPI Gateway, ChatRequest và ChatResponse. Các lớp biên này mô tả phân tiếp xúc giữa người dùng, frontend và backend.

3.4.2. Xác định thuộc tính và phương thức cho các lớp

Các lớp trong hệ thống được chia theo ba nhóm Boundary - Control - Entity để phù hợp với cách phân tích hướng đối tượng. Cách chia này giúp phân biệt lớp nào dùng để giao tiếp với người dùng, lớp nào xử lý logic và lớp nào biểu diễn dữ liệu nghiệp vụ.

a) Nhóm lớp thực thể

Product/CatalogProduct là lớp trung tâm của catalog. Thuộc tính chính gồm code, name, category, brand, price, price_value, context, specs, source_url, fetched_at, price_valid_until và image_url. Phương thức chính gồm to_dict(), image_source() và các hàm chuẩn hóa phục vụ hiển thị hoặc grounding.

Category biểu diễn danh mục sản phẩm, ví dụ Laptop hoặc Mobile Phone. Lớp này có các thuộc tính category_id, name, slug và có quan hệ một-nhiều với Product.

Brand biểu diễn thương hiệu sản phẩm. Thuộc tính chính gồm brand_id, name và slug. Một thương hiệu có thể có nhiều sản phẩm.

ProductSpec biểu diễn thông số kỹ thuật của sản phẩm. Thuộc tính gồm spec_key, spec_value, normalized_key, unit và confidence. Lớp này giúp hệ thống phân biệt RAM, SSD, CPU, GPU, màn hình hoặc pin thay vì chỉ lưu toàn bộ thông số trong một đoạn mô tả.

ProductImage lưu thông tin ảnh sản phẩm, bao gồm image_url, local_path, image_source và trạng thái ảnh chính. Lớp này giúp frontend hiển thị ảnh ổn định ngay cả khi thiếu ảnh cục bộ.

EvidenceFact hoặc EvidenceRef biểu diễn dữ kiện được phép dùng trong câu trả lời AI. Thuộc tính gồm product_code, fact_key, fact_text, source_url, valid_until và confidence. Đây là cơ sở để Verifier kiểm tra claim của AI.

ConversationSession, ChatMessage và AdvisorRun biểu diễn dữ liệu hội thoại. ConversationSession lưu state tổng quát của phiên; ChatMessage lưu nội dung người dùng và assistant; AdvisorRun lưu từng lượt xử lý của harness, bao gồm intent, query_frame, response_mode, verification_status và latency.

VerifierIssue và TraceEvent biểu diễn dữ liệu kiểm chứng và truy vết. VerifierIssue lưu các lỗi hoặc cảnh báo như missing_field, unsupported_claim, answer_cards_mismatch. TraceEvent lưu các sự kiện phase-level như plan, retrieve, verify, recover và commit.

b) Nhóm lớp điều khiển

CatalogService chịu trách nhiệm nạp catalog, tìm sản phẩm, lấy sản phẩm theo mã, resolve sản phẩm được nhắc đến trong câu hỏi và serialize dữ liệu cho frontend. Các phương thức quan trọng gồm search(), get(), resolve_product(), resolve_products(), resolve_constraints() và serialize_many().

ConversationPlanner xác định ý định hội thoại của người dùng. Lớp này phân biệt các tình huống như tìm kiếm sản phẩm, hỏi chi tiết, so sánh sản phẩm, loại bỏ candidate, hỏi sản phẩm tương tự hoặc đổi chủ đề.

QueryFrame hoặc QueryConstraints biểu diễn câu hỏi tự nhiên dưới dạng có cấu trúc. Các thuộc tính chính gồm category, brands, price_intent, cpu_filters, gpu_filters, use_case và requested_attributes. Đây là cầu nối giữa ngôn ngữ tự nhiên và truy vấn catalog.

ProductResolver xử lý các tham chiếu như mã sản phẩm, tên model, “mẫu đó”, “con thứ hai” hoặc “sản phẩm vừa tư vấn”. Nếu thiếu ProductResolver, hệ thống rất dễ trả lời sai trong hội thoại nhiều lượt.

AdvisorHarness hoặc HarnessRuntime là lớp điều phối toàn bộ quy trình AI Advisor. Lớp này bắt đầu lượt xử lý, ghi evidence, gọi verifier, xử lý recovery và kết thúc run với trạng thái thành công hoặc safe-degraded.

AIService tạo hoặc diễn đạt câu trả lời. Trong chế độ deterministic, hệ thống có thể tạo câu trả lời dựa trên catalog mà không cần external LLM. Khi external AI được bật, AIService vẫn phải nhận candidates và evidence từ harness.

Verifier kiểm tra câu trả lời trước khi trả về frontend. Các phương thức quan trọng gồm verify_response(), check_missing_fields(), check_text_card_match(), check_constraints() và check_strong_claim().

TraceCollector ghi lại các phase của một lượt xử lý. Nhờ TraceCollector, developer có thể biết hệ thống đã route intent nào, retrieve sản phẩm nào, verifier phát hiện lỗi gì và response cuối cùng được commit ra sao.

c) Nhóm lớp biên

ProductGrid và ProductCard là lớp biên phía frontend dùng để hiển thị danh sách sản phẩm và từng card sản phẩm. ProductCard sử dụng display_specs để ưu tiên hiển thị các thông số liên quan đến câu hỏi của người dùng.

CopilotDrawer là lớp biên hội thoại AI. Lớp này nhận input từ người dùng, gửi request đến API chat, hiển thị answer và render các product cards liên quan trong khung chat.

FastAPI Gateway là lớp biên phía backend. Nó nhận request từ frontend qua các endpoint sản phẩm, ảnh, chat, health và metrics. Gateway không chứa toàn bộ logic AI mà chuyển request tới các service phù hợp.

ChatRequest và ChatResponse là hai lớp dữ liệu biên giữa frontend và backend. ChatRequest chứa message, history và conversation_state. ChatResponse chứa answer_text, products, related_products, verification, missing_fields, conversation_state và decision_trace.

3.5. Kết luận chương

Chương này đã trình bày quá trình phân tích và thiết kế hệ thống AURA AI Sales Advisor theo hướng gần với tài liệu đồ án mẫu nhưng hạn chế sử dụng bảng. Nội dung chương bắt đầu từ mô tả hệ thống bằng ngôn ngữ tự nhiên, sau đó xác định yêu cầu, tác nhân, use case tổng quan, use case chi tiết, kịch bản use case, kiến trúc, biểu đồ lớp và lược đồ cơ sở dữ liệu.

Về phân tích chức năng, hệ thống được xác định gồm hai nhóm nghiệp vụ chính: storefront thương mại điện tử và AI Advisor có kiểm soát. Về thiết kế, hệ thống được chia thành frontend, API, domain/harness layer và data layer. Biểu đồ lớp cho thấy các lớp boundary, control và entity chính, trong đó các lớp như CatalogService, QueryFrame, ProductResolver, EvidenceRef, Verifier và TraceCollector là phần cốt lõi giúp AI Advisor không trả lời tự do.

Phần cơ sở dữ liệu cũng được trình bày theo hướng trung thực: bản prototype hiện dùng CSV catalog làm nguồn dữ liệu chính, còn lược đồ cơ sở dữ liệu quan hệ được đề xuất cho phiên bản production. Thiết kế này giúp hệ thống có thể mở rộng từ một prototype học thuật thành một hệ thống có khả năng lưu vết, kiểm chứng và đánh giá agent trong tương lai.

CHƯƠNG 4. CÀI ĐẶT VÀ KIỂM THỬ

Chương này trình bày quá trình cài đặt và kiểm thử hệ thống AURA AI Sales Advisor theo hướng bám sát cấu trúc báo cáo đề án tốt nghiệp: mô tả môi trường triển khai, cấu trúc mã nguồn, các thành phần đã cài đặt, giao diện hệ thống và các bước kiểm thử. Khác với một website thương mại điện tử thông thường, phần cài đặt của AURA cần chứng minh thêm lỗi AI Advisor được kiểm soát bằng harness, bao gồm trạng thái hội thoại, QueryFrame, truy xuất catalog, evidence, verifier, response contract và trace.

Cách trình bày trong chương này ưu tiên minh chứng triển khai bằng hình chụp giao diện, hình chụp API/terminal, sơ đồ luồng xử lý và mô tả ngắn từng thành phần. Các vị trí có ghi “cần thay bằng ảnh chụp thực tế” là nơi sinh viên nên chèn screenshot lấy trực tiếp từ máy chạy đề án trước khi nộp bản cuối.

4.1. Môi trường triển khai ứng dụng

Hệ thống được triển khai thử nghiệm trên môi trường local phục vụ mục tiêu đề án tốt nghiệp. Môi trường này đủ để kiểm tra các chức năng catalog, API, giao diện người dùng, AI Advisor và các test tự động. Thành phần backend được triển khai bằng Python/FastAPI; thành phần frontend được triển khai bằng React, Vite và TypeScript; dữ liệu sản phẩm trong phiên bản prototype được đọc từ catalog dạng CSV.

Hệ điều hành phát triển: Windows hoặc Linux đều có thể chạy được nếu cài đủ Python, Node.js và npm.

Backend: Python 3.10 trở lên, FastAPI, Pydantic, Uvicorn, pytest.

Frontend: Node.js 18 trở lên, npm, React, Vite, TypeScript.

Dữ liệu: product_catalog_real.csv, product_images.csv và thư mục ảnh sản phẩm.

Cấu hình local: file .env tạo từ .env.example, không commit API key hoặc thông tin nhạy cảm.

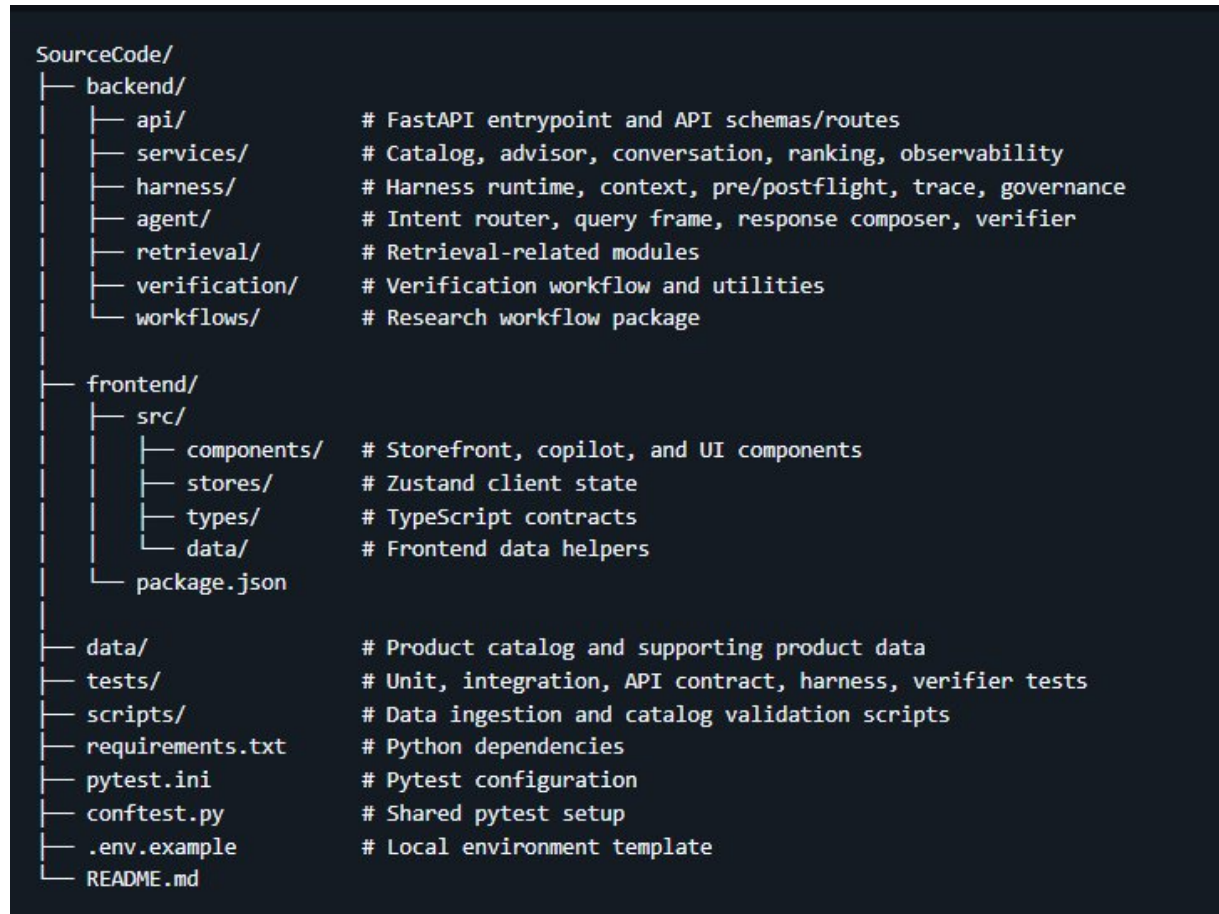
Một điểm đáng chú ý là chế độ chạy mặc định của hệ thống dùng deterministic catalog logic, tức hệ thống vẫn có thể tư vấn dựa trên catalog mà không bắt buộc phải bật khóa external LLM. Cách này phù hợp với đề án vì giúp kiểm thử ổn định, giảm phụ thuộc quota API và làm rõ đóng góp của harness thay vì chỉ phụ thuộc vào mô hình ngôn ngữ.

```
pip install -r requirements.txt
python -m uvicorn backend.api.main:app --host 127.0.0.1 --port 8000 --reload --reload-dir backend

cd frontend
npm install
npm run dev
```

4.2. Cấu trúc mã nguồn hệ thống

Mã nguồn AURA được tổ chức theo kiến trúc tách lớp. Thư mục backend chứa API, service, harness, agent, retrieval và verification. Thư mục frontend chứa các component giao diện, store trạng thái client, type contract và helper dữ liệu. Thư mục data lưu catalog sản phẩm và dữ liệu ảnh. Thư mục tests lưu các nhóm kiểm thử phục vụ API contract, harness, verifier và product reference resolution.



Hình 4.1. Cấu trúc mã nguồn hệ thống AURA

Cách tổ chức này giúp báo cáo không bị biến thành mô tả repo rời rạc. Mỗi thư mục tương ứng với một lớp trong kiến trúc thiết kế: frontend đảm nhiệm giao diện, backend/api đảm nhiệm ranh giới hệ thống, services xử lý nghiệp vụ catalog, harness điều phối AI Advisor, agent chứa các module quyết định như intent router, query frame và verifier, còn tests phục vụ kiểm chứng hành vi.

backend/api: định nghĩa FastAPI app, schema request/response và các route chính.

backend/services: xử lý catalog, hội thoại, ranking và observability.

backend/harness: runtime, preflight, postflight, trace và governance.

backend/agent: intent router, QueryFrame, product resolver, evidence, response composer và verifier.

frontend/src/components: giao diện catalog, ProductCard, CopilotDrawer và inline product cards.

tests: các test tự động cho API contract, harness runtime, resolver và verifier.

4.3. Cài đặt backend API

Backend đóng vai trò là cổng giao tiếp giữa frontend, catalog và AI Advisor. Thành phần này được triển khai bằng FastAPI. Trong file backend/api/main.py, hệ thống khởi tạo FastAPI app, bật CORS cho frontend local, định nghĩa các schema ChatRequest, ChatResponse và triển khai các endpoint chính.

4.3.1. Các endpoint chính

Endpoint /health dùng để kiểm tra trạng thái backend, số lượng sản phẩm catalog đã load và trạng thái AI service. Endpoint /metrics dùng để lấy các số liệu quan sát của agent, harness và shadow mode. Nhóm endpoint /api/products phục vụ duyệt catalog, tìm kiếm/lọc sản phẩm, lấy sản phẩm nổi bật, lấy chi tiết sản phẩm và lấy ảnh sản phẩm. Endpoint /api/chat là endpoint quan trọng nhất vì nó nhận câu hỏi người dùng, kích hoạt pipeline AI Advisor và trả về ChatResponse có cấu trúc.

```
GET /health
GET /metrics
GET /api/products/featured
GET /api/products?q=&category=&brand=&limit=&offset=
GET /api/products/{code}
GET /api/products/{code}/image
POST /api/chat
```

Việc tách endpoint rõ ràng giúp frontend không cần biết chi tiết bên trong AI Advisor. Frontend chỉ gọi API và render theo contract. Backend chịu trách nhiệm validate request, truy xuất catalog, điều phối harness, kiểm chứng câu trả lời và đóng gói response.

4.3.2. Schema ChatRequest và ChatResponse

Request của chức năng chat gồm message, history và conversation_state. Cách thiết kế này cho phép hệ thống không chỉ nhận câu hỏi hiện tại mà còn nhận bối cảnh hội thoại từ frontend. Response trả về không chỉ là text mà là một object có nhiều trường phục vụ giao diện và kiểm chứng.

message: câu hỏi tiếng Việt người dùng nhập vào.

history: lịch sử hội thoại ngắn, gồm role, text và product_codes.

`conversation_state`: trạng thái quyết định ở lượt trước, ví dụ `category`, ngân sách, `active product`, `candidate codes`.

`answer_text/text`: nội dung trả lời hiển thị cho người dùng.

`related_products/products`: danh sách sản phẩm liên quan để render product cards.

`missing_fields` và `warnings`: các cảnh báo khi catalog thiếu dữ liệu hoặc câu trả lời cần thận trọng.

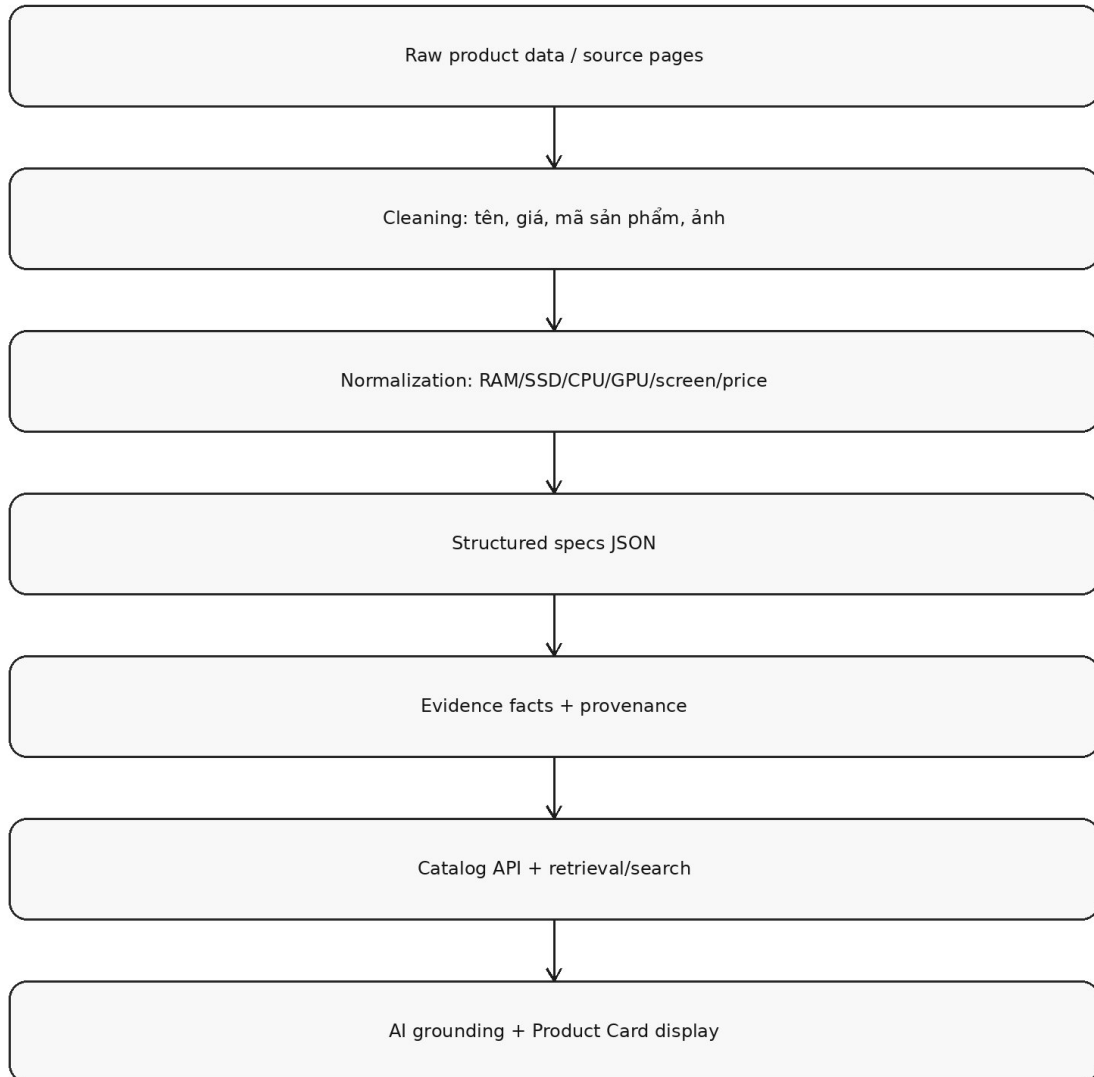
`verification`: kết quả kiểm chứng câu trả lời.

`decision_trace`: trace khi bật chế độ `debug/expose trace`.

4.4. Cài đặt dữ liệu catalog và xử lý sản phẩm

Catalog là nguồn sự thật chính của hệ thống. Trong phiên bản hiện tại, dữ liệu sản phẩm được đọc từ file `product_catalog_real.csv`. Nếu file dữ liệu thật không tồn tại, hệ thống có thể fallback sang `product_catalog_clean.csv`. Thành phần `CatalogService` chịu trách nhiệm load catalog, tạo chỉ mục theo product code, đọc ảnh sản phẩm, phân tích query, lọc sản phẩm và serialize dữ liệu trả về frontend.

Quy trình xử lý dữ liệu catalog phục vụ AI Advisor



Hình 4.2. Pipeline xử lý dữ liệu catalog phục vụ hiển thị và AI grounding

Mỗi sản phẩm trong catalog được ánh xạ thành `CatalogProduct` với các trường như `code`, `category`, `brand`, `price`, `context`, `specs`, `title`, `source_url`, `fetch_at`, `price_valid_until`, `external_image_url` và `spec_provenance_json`. Phương thức `to_dict()` chuyển sản phẩm sang dữ liệu JSON gồm `id`, `code`, `name`, `category`, `brand`, `price`, `price_value`, `description`, `specs`, `image_url`, `source_url`, `fetch_at` và `spec_provenance`.

Ở lớp xử lý dữ liệu, hệ thống có các hàm chuẩn hóa tiếng Việt, chuẩn hóa giá, parse ngân sách, phát hiện CPU/GPU, trích xuất thông số và nhận diện các trường như RAM, SSD, pin, tần số quét, trọng lượng, kháng nước và bảo hành nếu có trong dữ liệu nguồn. Điều này giúp catalog không chỉ phục vụ hiển thị mà còn phục vụ retrieval và verifier.

4.4.1. Truy xuất và lọc catalog

CatalogService hỗ trợ các thao tác `get()`, `search()`, `resolve_product()`, `resolve_products()`, `analyze_query()` và `resolve_constraints()`. Khi người dùng hỏi hoặc lọc sản phẩm, hệ thống không chỉ tìm kiếm theo từ khóa mà còn nhận diện category, brand, ngân sách, CPU, GPU, mục tiêu sử dụng và ngữ cảnh hội thoại. Đây là cơ sở để QueryFrame và ProductResolver hoạt động ổn định.

Tìm sản phẩm theo Product Code/SKU trước để tránh nhầm model.

Phát hiện category như Laptop hoặc Mobile Phone từ câu hỏi.

Phát hiện brand, ngân sách dạng “dưới 20 triệu”, “tầm 25 triệu”, “20 đến 30 triệu”.

Phát hiện CPU/GPU như i5, i7, Ryzen 5, RTX, GTX, Radeon RX.

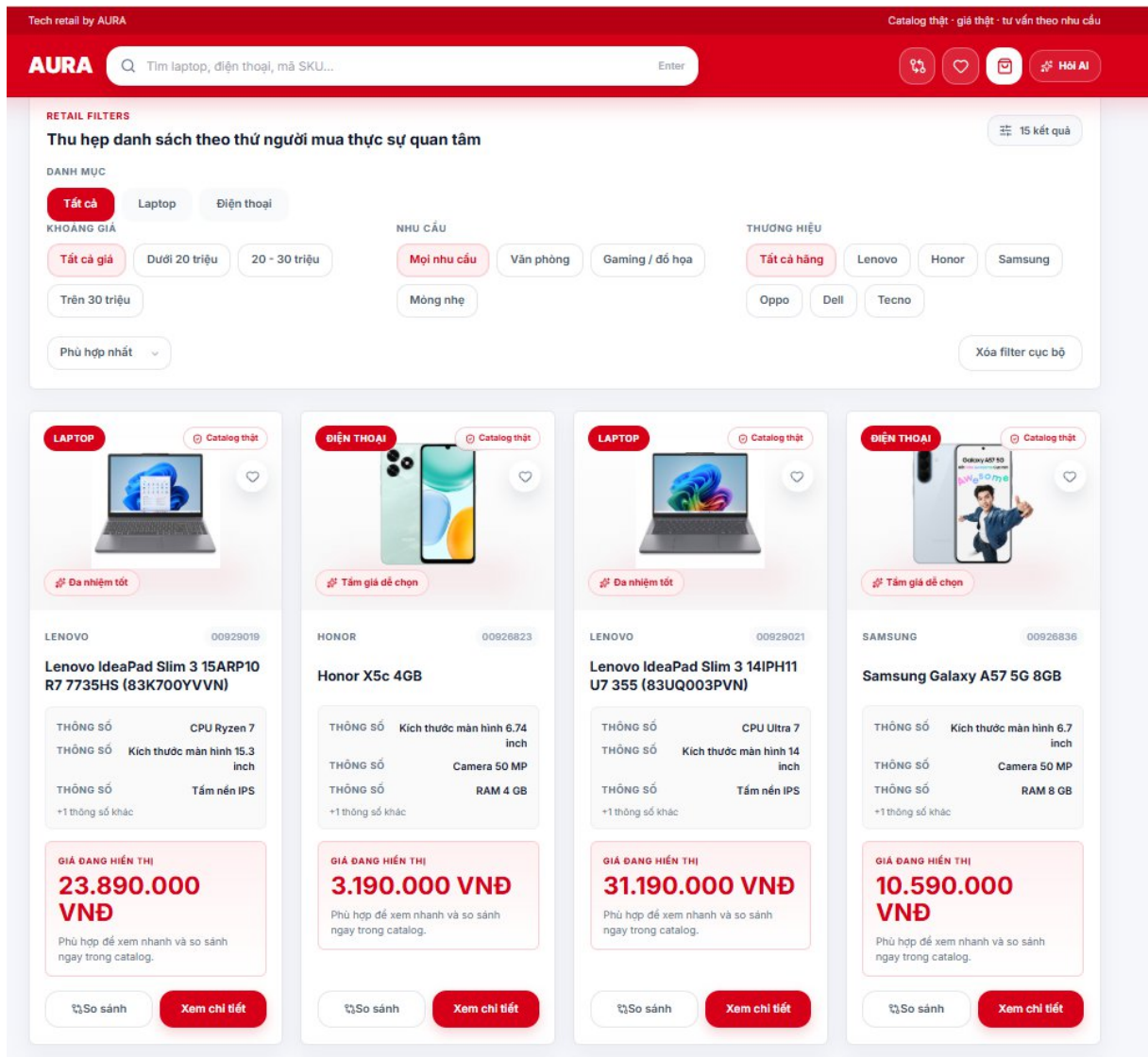
Kế thừa category hoặc budget từ conversation_state khi người dùng hỏi tiếp.

4.5. Cài đặt frontend và giao diện hệ thống

Frontend được xây dựng theo hướng component-based. Các component chính gồm ProductGrid, ProductCard, InlineProductCard, CopilotDrawer, ChatMessageText và các store quản lý trạng thái. Giao diện không chỉ hiển thị catalog mà còn phải thể hiện được sự đồng bộ giữa câu trả lời AI và product cards.

4.5.1. Giao diện catalog sản phẩm

ProductGrid hiển thị danh sách sản phẩm và các công cụ lọc theo danh mục, khoảng giá, nhu cầu sử dụng và thương hiệu. Giao diện có các trạng thái loading, loading more, empty state và lỗi API. Người dùng có thể lọc laptop/điện thoại, lọc theo giá dưới 20 triệu, 20-30 triệu, trên 30 triệu, lọc theo nhu cầu văn phòng, gaming/đồ họa hoặc mỏng nhẹ.

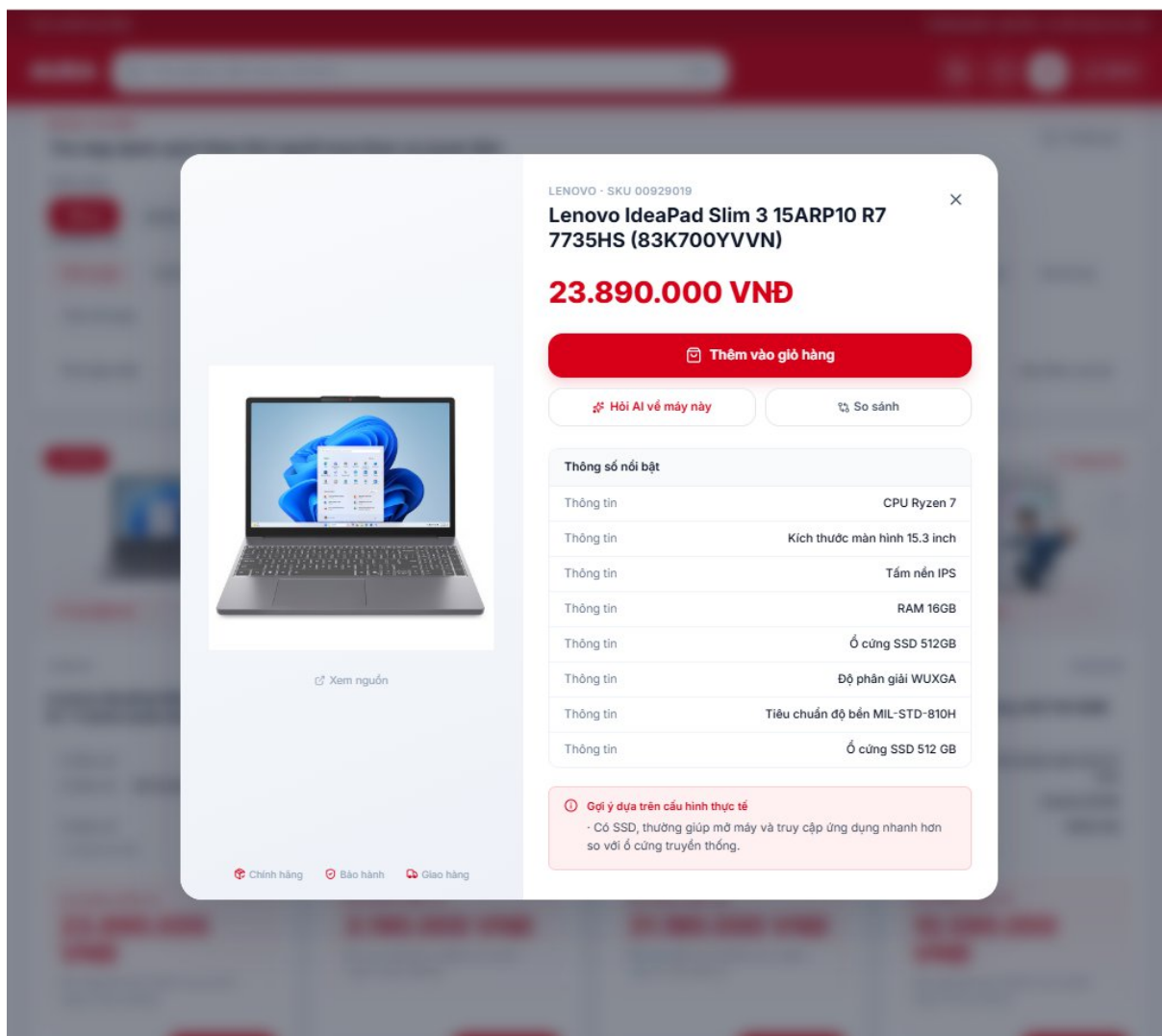


Hình 4.3. Giao diện catalog sản phẩm

Trong báo cáo chính thức, hình này cần được thay bằng ảnh chụp màn hình ProductGrid khi backend và frontend đang chạy. Ảnh nên thể hiện rõ danh sách product cards, filter panel, số kết quả đang hiển thị và nút xem thêm sản phẩm.

4.5.2. Giao diện ProductCard và chi tiết sản phẩm

ProductCard là thành phần giúp người dùng đối chiếu câu trả lời AI với dữ liệu catalog. Một product card cần thể hiện tối thiểu tên sản phẩm, mã sản phẩm, thương hiệu, giá, ảnh và một số thông số nổi bật. Trong AURA, sản phẩm còn có display_specs để ưu tiên hiển thị các thông số liên quan trực tiếp đến câu hỏi người dùng, ví dụ RAM/SSD khi người dùng hỏi về cấu hình lưu trữ.

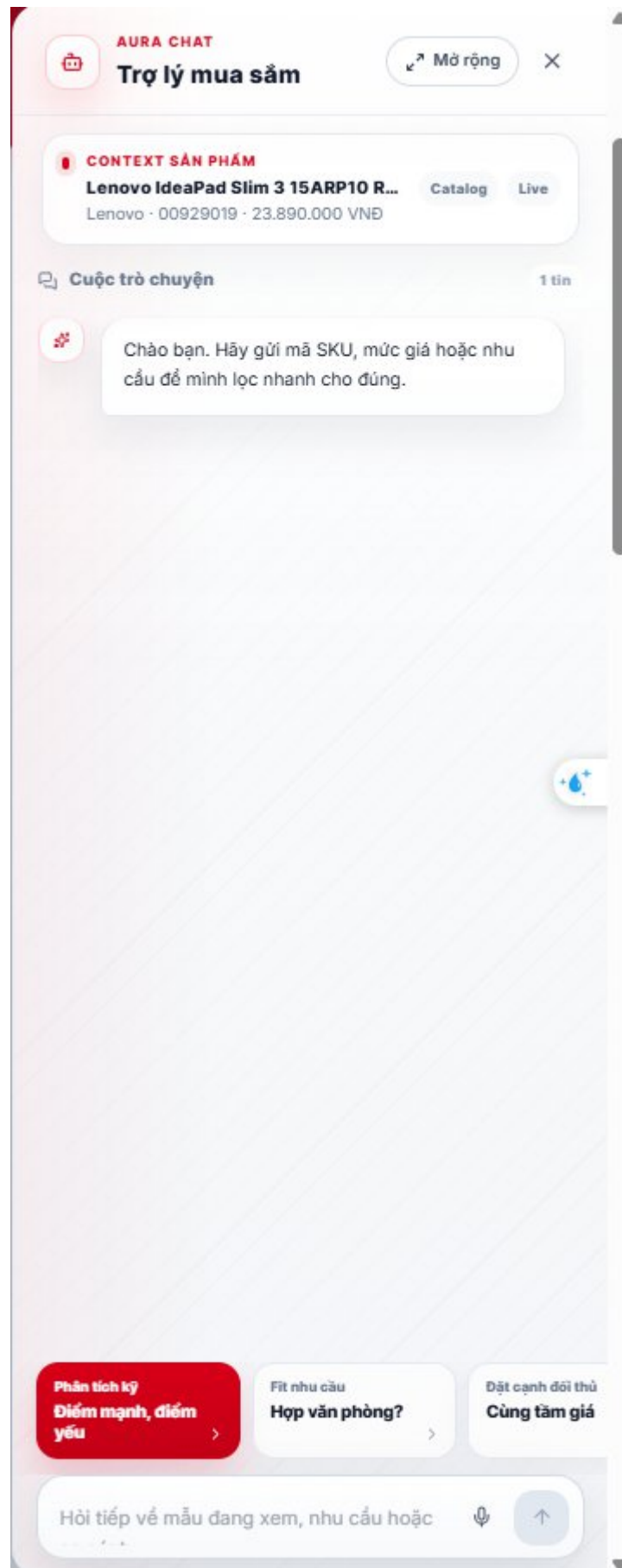


Hình 4.4. Giao diện ProductCard

Khi chụp ảnh thay thế, nên chọn một sản phẩm có đủ tên, mã, giá và specs. Nếu có thể, nên chụp cùng một sản phẩm xuất hiện trong câu trả lời AI để chứng minh text và card đồng bộ.

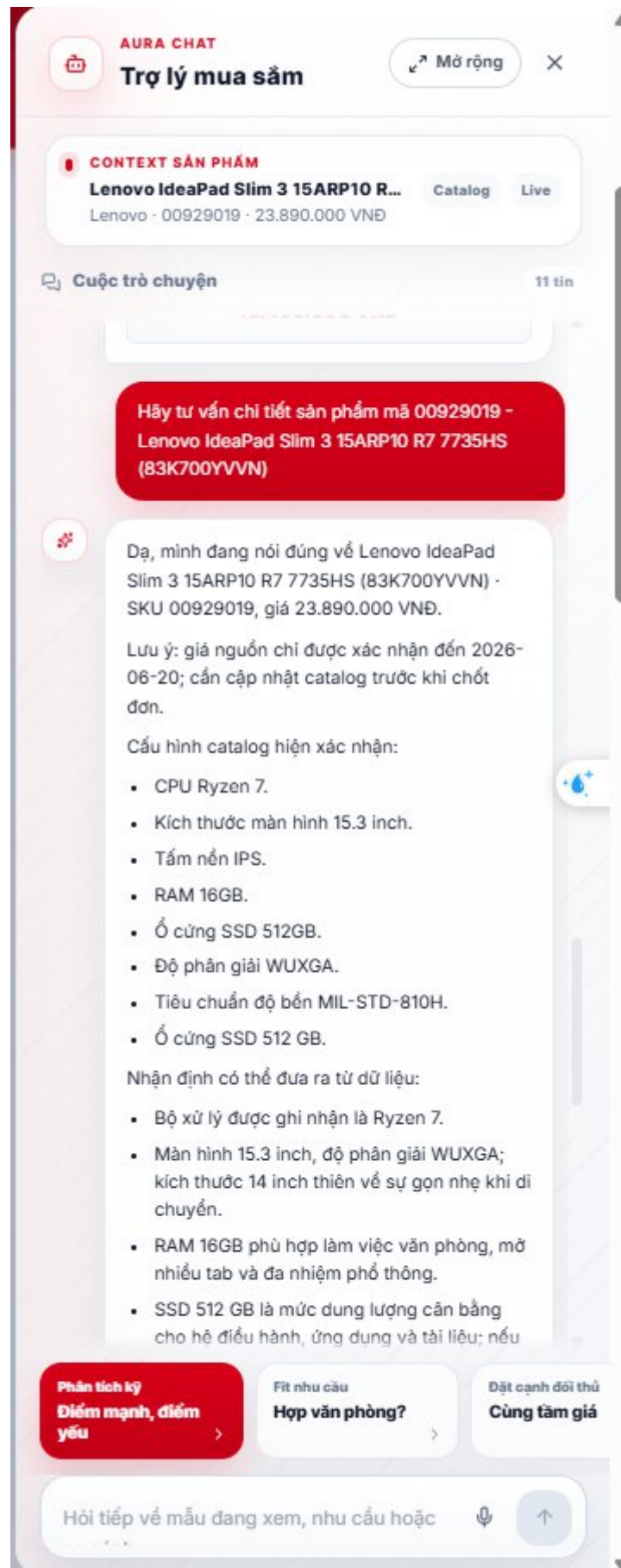
4.5.3. Giao diện CopilotDrawer

CopilotDrawer là giao diện hội thoại của AI Advisor. Thành phần này có nút “Hỏi AI”, khung chat dạng drawer, các quick prompts, context panel, danh sách tin nhắn, inline product cards, input hỗ trợ Enter để gửi và Shift + Enter để xuống dòng. Khi người dùng đang chọn một sản phẩm, CopilotDrawer hiển thị trạng thái Focused; khi chưa có sản phẩm cụ thể, hệ thống ở trạng thái Broad và sẽ bám theo ngữ cảnh hội thoại gần nhất.



Hình 4.5. Giao diện CopilotDrawer

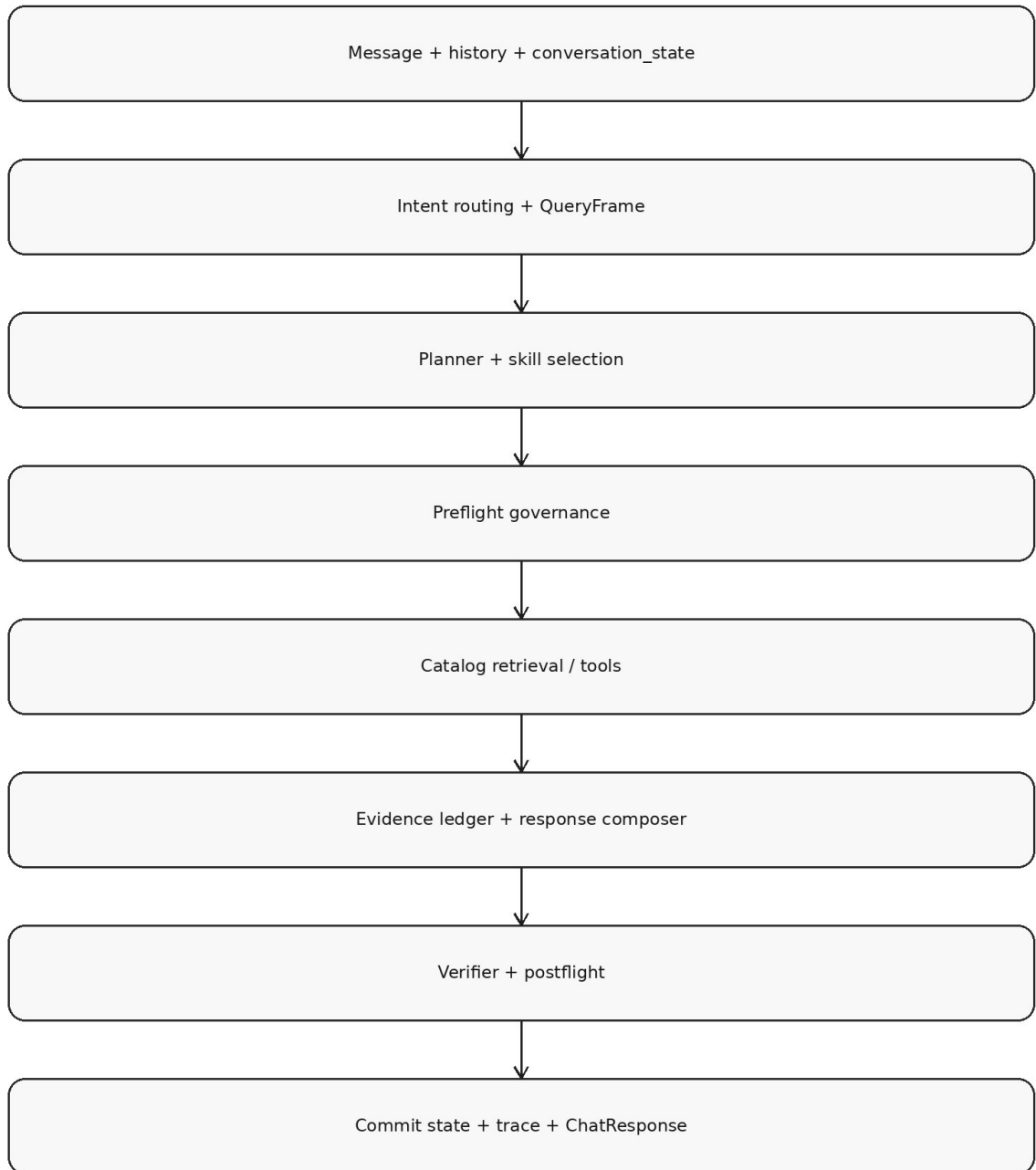
CopilotDrawer cũng có các gợi ý nhanh như “Laptop học tập dưới 20 triệu”, “So sánh laptop gaming tầm trung”, “Điện thoại chụp ảnh đẹp”. Khi người dùng đang chọn một sản phẩm, hệ thống có các gợi ý theo context như “Phân tích kỹ”, “Hợp văn phòng?” hoặc “So sánh cùng tầm giá”. Đây là chi tiết giao diện quan trọng vì nó chứng minh AI Advisor được đặt trong storefront, không phải chatbot tách rời.



4.6. Cài đặt AI Advisor và Harness Engineering

Đây là phần cài đặt quan trọng nhất của đề án. AI Advisor không được thiết kế như một chatbot trả lời tự do. Mỗi request từ /api/chat được xử lý qua một chuỗi pha có kiểm soát: perceive, plan, preflight, retrieve/execute, postflight, recover và commit. Cách tổ chức này giúp hệ thống có thể debug lỗi theo từng pha thay vì chỉ đọc câu trả lời cuối cùng.

Luồng xử lý một lượt hỏi đáp AI Advisor



Hình 4.7. Luồng xử lý AI Advisor từ câu hỏi tới response contract

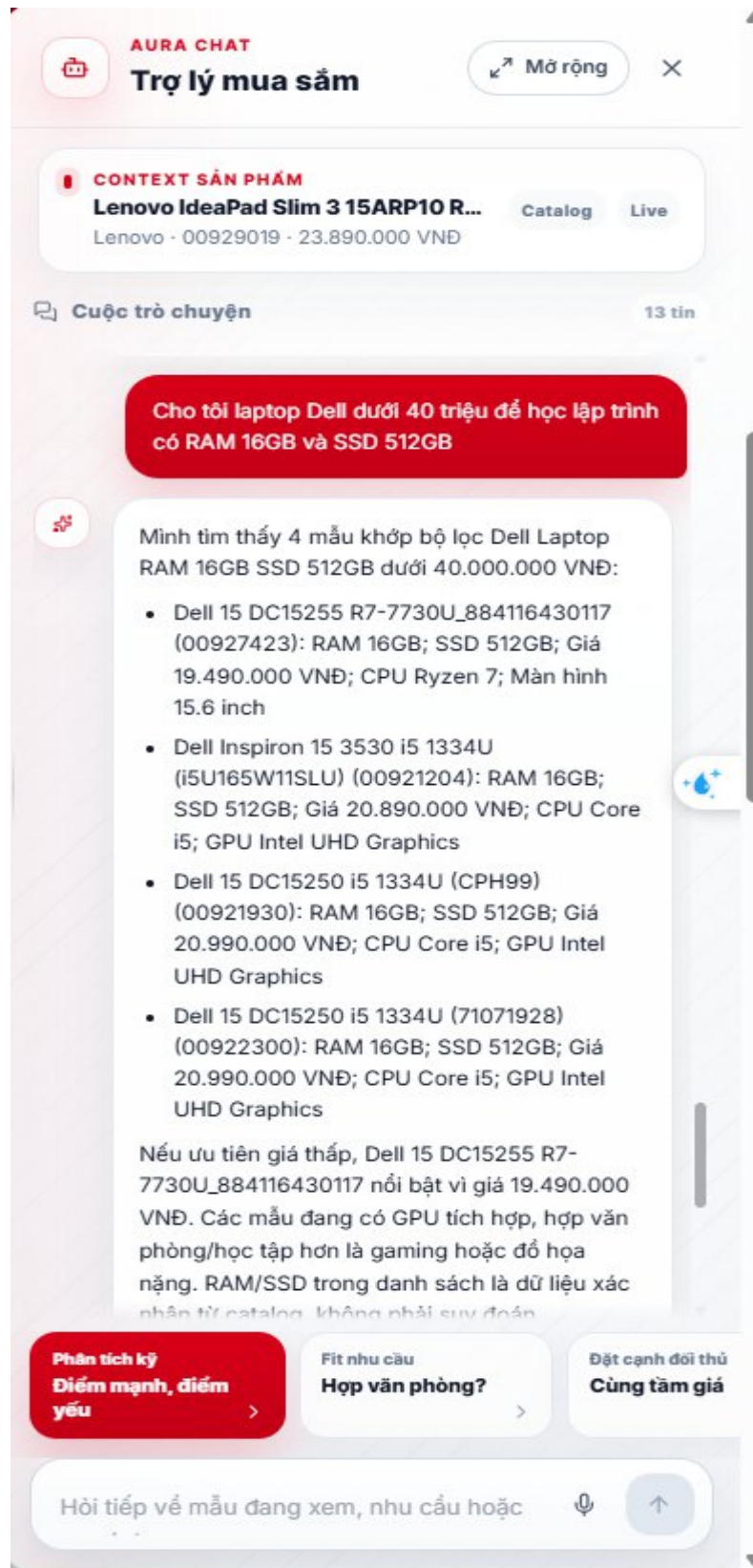
4.6.1. Pha perceive - nạp state và kiểm tra catalog revision

Khi nhận ChatRequest, hệ thống tạo TraceCollector với request_id, hash của user message và execution budget. Sau đó, backend load catalog, tính catalog_revision và nạp conversation_state từ request. Nếu state đã hết hạn hoặc stale so với catalog hiện tại, hệ thống có thể reset hoặc ghi trace để tránh sử dụng ngữ cảnh cũ. Pha này giúp AI Advisor xử lý hội thoại nhiều lượt nhưng vẫn tránh việc giữ state sai quá lâu.

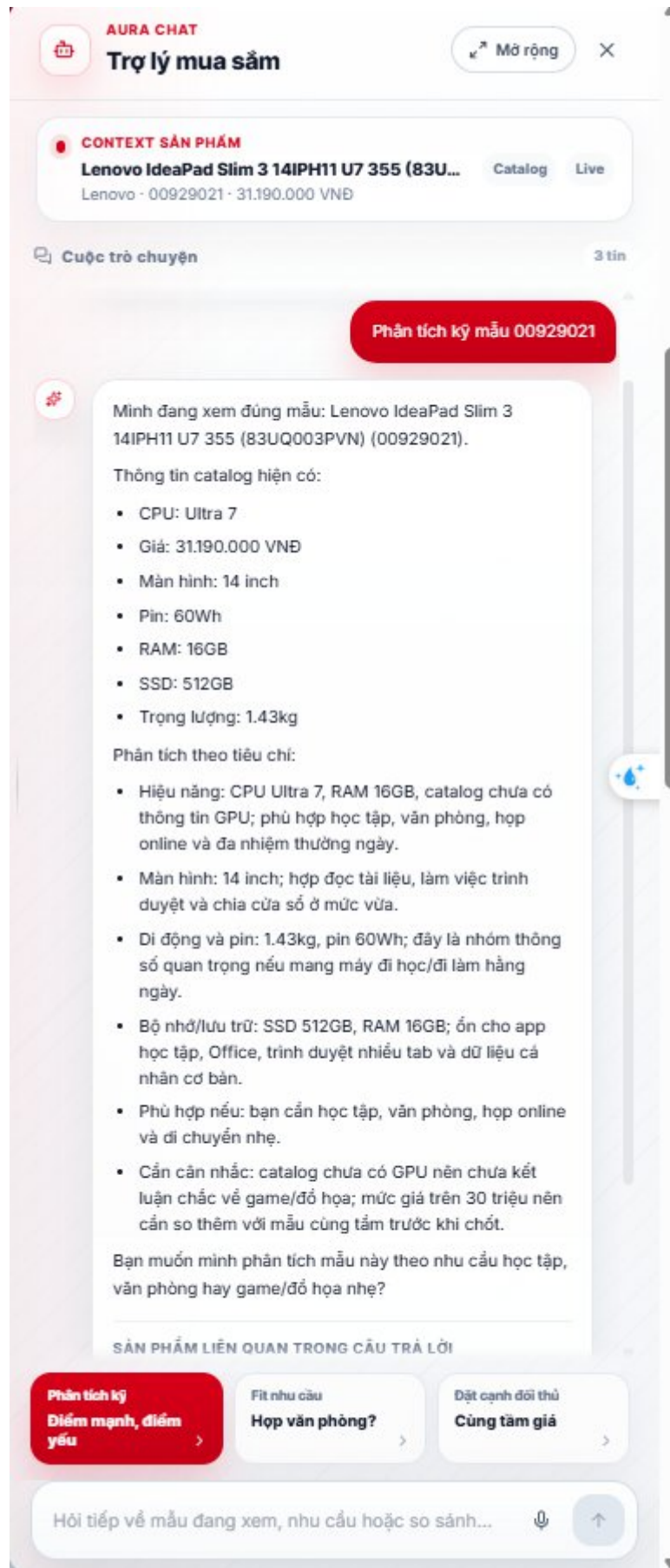
4.6.2. Pha plan - xác định intent và QueryFrame

Sau khi có state, hệ thống sử dụng `advisor_harness` và `ConversationPlanner` để xác định `dialogue_act`. Các intent quan trọng gồm tìm kiếm sản phẩm mới, hỏi chi tiết sản phẩm, hỏi tiếp theo ngữ cảnh, so sánh sản phẩm, yêu cầu sản phẩm rẻ hơn hoặc từ chối candidate trước đó. Từ intent, hệ thống xây dựng `QueryConstraints/QueryFrame` gồm `category`, `brand`, `price_intent`, `CPU/GPU filters`, `use_case` và `goal`.

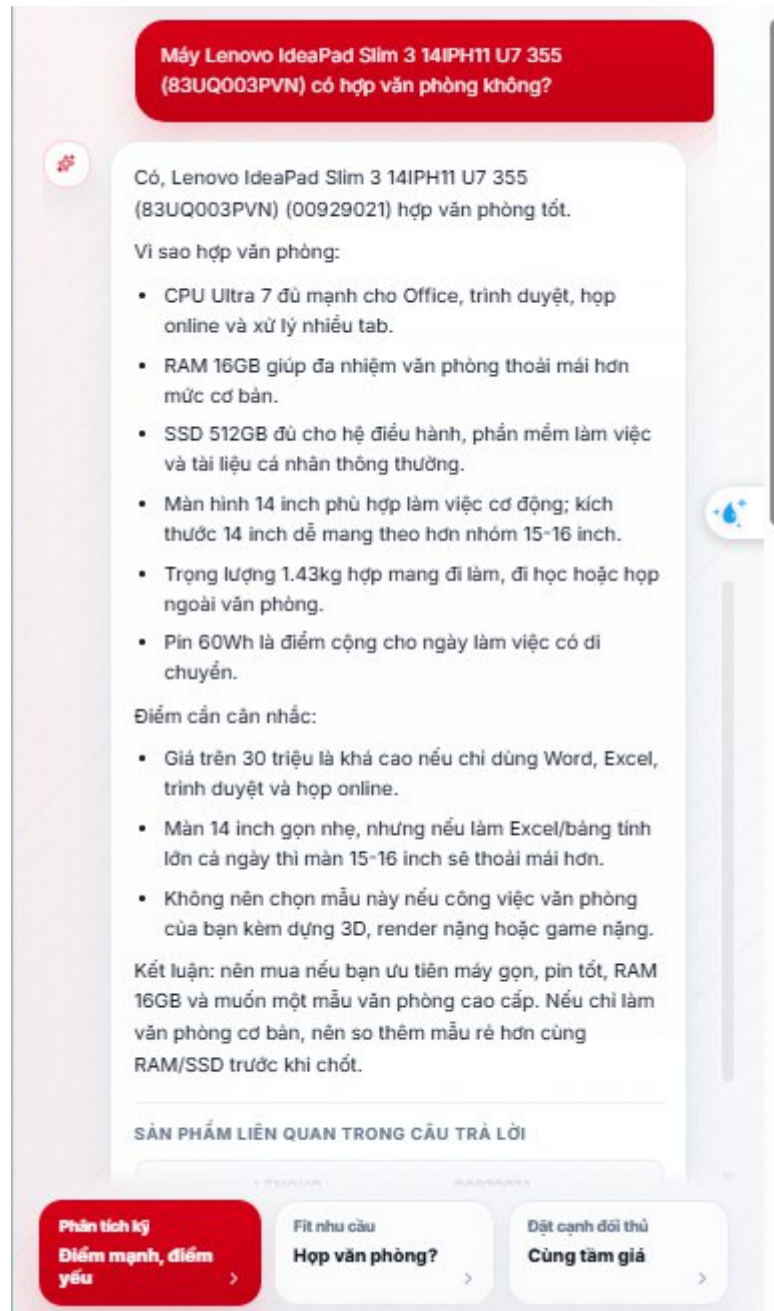
Ví dụ với câu “Cho tôi laptop Dell dưới 20 triệu để học lập trình”, `QueryFrame` có thể gồm `category = Laptop`, `brand = Dell`, `price maximum = 20.000.000` và `use_case = programming`. Việc chuyển câu hỏi tự nhiên thành `QueryFrame` giúp hệ thống truy xuất catalog theo ràng buộc rõ ràng thay vì đưa toàn bộ câu hỏi vào LLM.



Hình 4.8. Minh họa câu hỏi cấu hình RAM/SSD và display_specs



Hình 4.9. Minh họa câu hỏi phân tích kỹ 1 mẫu máy



Hình 4.10. Minh họa câu hỏi có phù hợp cho văn phòng không

So sánh mẫu 00929021 với các máy cùng tầm giá

Mình so sánh nhanh theo dữ liệu catalog hiện có:

Tiêu chí	Lenovo IdeaPad Slim 3 14IPH11 U7 355 (83UQ003PVN) (00929021)	HP EliteBook 6 G1i Core Ultra 5 225H (BQ9M9PT)
Giá	31.190.000 VNĐ	30.990.000 VNĐ
CPU	Ultra 7	Core Ultra 5
RAM	16GB	16GB
SSD	512GB	512GB
GPU	Chưa có dữ liệu	Chưa có dữ liệu
Màn hình	14 inch	14 inch

Nếu ưu tiên giá thấp, HP EliteBook 6 G1i Core Ultra 5 225H (BQ9M9PT) lợi hơn.

SẢN PHẨM LIÊN QUAN TRONG CÂU TRẢ LỜI

LENOVO 00929021

Lenovo IdeaPad Slim 3 14IPH11 U7 355...

Giá 31.190.000 VNĐ · CPU Ultra 7

31.190.000 VNĐ

+ Thêm

Phân tích kỹ

Điểm mạnh, điểm yếu

>

Fit nhu cầu

Hợp văn phòng?

>

Đặt cạnh đối thủ

Cùng tầm giá

>

Hỏi tiếp về mẫu đang xem, nhu cầu hoặc so sánh...

Hình 4.11. Minh họa câu hỏi so sánh cùng 1 tầm giá

4.6.3. Pha retrieve/execute - truy xuất catalog và sinh response

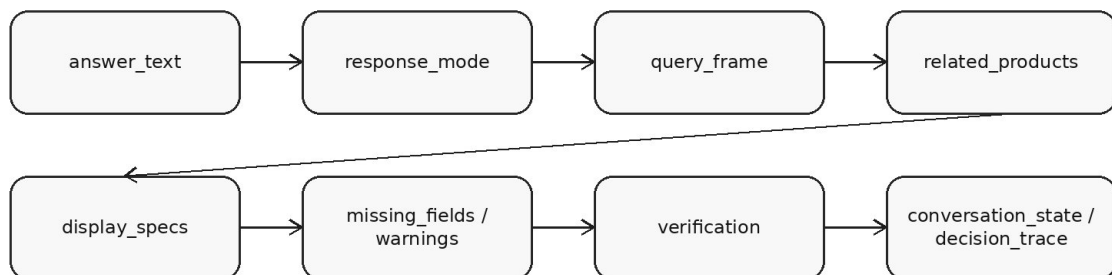
Ở pha retrieve/execute, hệ thống sử dụng CatalogService để tìm sản phẩm phù hợp với QueryFrame hoặc resolve sản phẩm từ context. Nếu câu hỏi là follow-up như “mẫu đó pin sao?”, ProductResolver sẽ ưu tiên active_product_code hoặc candidate_codes từ state. Nếu câu hỏi là so sánh, hệ thống lấy các sản phẩm đã được chọn hoặc vừa hiển thị trong hội thoại. Sau khi có matched_products, AIService hoặc response composer tạo câu trả lời có kèm product_codes, tools_used và nguồn dữ liệu.

Trong chế độ deterministic contract-first, backend có thể đi qua route_intent, build_query_frame, search_products hoặc get_product_field, build_evidence_ledger, compose_response, verify_response và domain_contract. Cách này làm rõ vai trò của harness: AI chỉ được sinh phản hồi dựa trên sản phẩm và facts được lấy từ catalog.

4.6.4. Evidence, Verifier và Response Contract

Evidence Ledger ghi lại các facts mà câu trả lời được phép sử dụng. Verifier kiểm tra câu trả lời trước khi commit response. Các lỗi cần kiểm soát gồm sản phẩm không tồn tại, claim không có evidence, thiếu field nhưng không công bố, sản phẩm không thỏa constraint và text-card mismatch. Nếu verifier hoặc postflight phát hiện lỗi nghiêm trọng, hệ thống chuyển sang safe_degrade hoặc yêu cầu người dùng làm rõ thay vì trả câu trả lời thiếu căn cứ.

Response contract giữa backend AI và frontend



Hình 4.12. Response Contract đồng bộ answer, product cards, verification và state

Response Contract của AURA gồm text/answer_text, response_mode, query_frame, related_products, missing_fields, warnings, tools_used, sources, products, verification, conversation_state, active_context, follow_up_question và decision_trace. Nhờ contract này, frontend có thể render câu trả lời cùng sản phẩm liên quan, đồng thời giữ lại state cho lượt hội thoại tiếp theo.

4.6.5. Trace và recovery

TraceCollector ghi lại các phase như perceive, plan, preflight, retrieve_execute, postflight, recover và commit. Khi bật EXPOSE_DECISION_TRACE hoặc debug mode, response có thể kèm decision_trace để phục vụ phân tích. Trace giúp trả lời các câu hỏi kỹ thuật như: hệ thống route intent nào, QueryFrame ra sao, tool nào được gọi, sản phẩm nào được chọn, verifier có phát hiện lỗi không và state sau cùng thay đổi thế nào.

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/chat' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "message": "Cho tôi laptop Dell dưới 20 triệu để học lập trình",
    "history": [
      {
        "role": "assistant",
        "text": "string",
        "product_codes": [
          "string"
        ]
      }
    ],
    "conversation_state": {
      "category": "string",
      "budget_target": 0,
      "budget_minimum": 0,
      "budget_maximum": 0,
      "goal": "string",
      "use_case": "string",
      "active_product_code": "string",
      "compared_codes": [
        "string"
      ],
      "compared_brands": [
        "string"
      ]
    }
  }'
```

```
{
  "text": "Mình tìm thấy 2 mẫu khớp bộ lọc Dell Laptop dưới 20.000.000 VNĐ:\n- Dell Pro 15 Essential PV15250 Core 3-100U (VKVKD) (00927778): Giá 14.490.000 VNĐ; CPU Core 3; RAM 8GB; SSD 512GB; Màn hình 15.6 inch 120Hz\n- Dell 15 DC15255 R7-7730U_884116430117 (00927423): Giá 19.490.000 VNĐ; CPU Ryzen 7; RAM 16GB; SSD 512GB; Màn hình 15.6 inch\n\nNếu ưu tiên giá thấp, Dell Pro 15 Essential PV15250 Core 3-100U (VKVKD) nổi bật vì giá 14.490.000 VNĐ.\n\nBạn muốn mình so sánh 2 mẫu đáng chú ý nhất theo giá, CPU/GPU và màn hình không?",
  "answer_text": "Mình tìm thấy 2 mẫu khớp bộ lọc Dell Laptop dưới 20.000.000 VNĐ:\n- Dell Pro 15 Essential PV15250 Core 3-100U (VKVKD) (00927778): Giá 14.490.000 VNĐ; CPU Core 3; RAM 8GB; SSD 512GB; Màn hình 15.6 inch 120Hz\n- Dell 15 DC15255 R7-7730U_884116430117 (00927423): Giá 19.490.000 VNĐ; CPU Ryzen 7; RAM 16GB; SSD 512GB; Màn hình 15.6 inch\n\nNếu ưu tiên giá thấp, Dell Pro 15 Essential PV15250 Core 3-100U (VKVKD) nổi bật vì giá 14.490.000 VNĐ.\n\nBạn muốn mình so sánh 2 mẫu đáng chú ý nhất theo giá, CPU/GPU và màn hình không?",
  "response_mode": "filtered_search_result",
  "query_frame": {
    "intent": "new_filtered_search",
    "exclude_product_codes": [],
    "inherit_from_last_query_frame": false,
    "constraints": {
      "category": "Laptop",
      "brand": "Dell",
      "min_price": null,
      "max_price": 20000000,
      "cpu_tier": null,
      "gpu_type": null,
      "ram_gb": null,
      "storage_gb": null,
      "screen_inches": null,
      "use_case": null
    },
    "requested_attributes": [
      "price_value"
    ]
  }
}
```

Hình 4.13. Minh họa decision_trace/harness trace

Cơ chế recovery được kích hoạt khi preflight fail, budget exhausted, postflight fail hoặc verifier không approve. Thay vì để lỗi lan tới người dùng, hệ thống có thể trả về

câu hỏi làm rõ, thông báo thiếu dữ liệu hoặc safe-degrade. Đây là điểm quan trọng để chứng minh AI Advisor được kiểm soát bằng kỹ thuật phần mềm chứ không chỉ bằng prompt.

4.7. Kiểm thử hệ thống

Kiểm thử trong AURA được thực hiện theo nhiều lớp. Với hệ thống e-commerce thông thường, kiểm thử chủ yếu tập trung vào API và giao diện. Với AURA, cần kiểm thử thêm các lớp AI Advisor như intent, QueryFrame, ProductResolver, Evidence, Verifier, Response Contract và Trace. Nguyên tắc đánh giá là không chỉ xem câu trả lời cuối cùng có vẻ đúng, mà phải kiểm tra toàn bộ quá trình tạo ra câu trả lời.

4.7.1. Kiểm thử API thủ công bằng curl

Trước khi kiểm thử chức năng AI Advisor, cần kiểm tra backend đã chạy và catalog đã load đúng. Các lệnh kiểm thử thủ công sau giúp xác nhận ba nhóm chức năng cơ bản: health check, product API và chat API.

```
curl http://127.0.0.1:8000/health

curl "http://127.0.0.1:8000/api/products?limit=2"

curl -X POST http://127.0.0.1:8000/api/chat \
-H "Content-Type: application/json" \
-d '{"message":"Cho tôi laptop Dell dưới 20 triệu","history":[],"conversation_state":null}'
```

Kết quả mong muốn của /health là status = ok, có số lượng catalog_products và trạng thái AI. Kết quả mong muốn của /api/products là danh sách sản phẩm có code, name, brand, price, price_value, specs và image_url. Kết quả mong muốn của /api/chat là ChatResponse có text/answer_text, products hoặc related_products, workflow_status, ai_mode, tools_used, verification và conversation_state.

4.7.2. Kiểm thử tự động backend

Theo cấu trúc dự án, nhóm test được chia thành các file kiểm thử tập trung. Có thể chạy focused regression tests để kiểm tra intent router và API contract runtime; sau đó chạy nhóm harness runtime và product reference resolution để kiểm tra các tình huống hội thoại có ngữ cảnh.

```
python -m pytest tests/test_intent_router_v2.py tests/test_api_contract_runtime.py -q

python -m pytest tests/test_harness_runtime.py tests/test_product_reference_resolution.py -q
```

Khi viết báo cáo cuối, không nên ghi số lượng test pass nếu chưa có log thực tế. Cách làm đúng là chạy lệnh trên máy nộp, chụp màn hình terminal và điền số liệu thật

vào báo cáo: tổng số test, số test pass, thời gian chạy và nếu có thì các test fail đã được sửa.

API contract test: kiểm tra /api/chat trả đúng schema và frontend có thể render.

Intent router test: kiểm tra các câu hỏi tiếng Việt được route đúng intent.

QueryFrame test: kiểm tra trích xuất category, brand, budget, RAM/SSD, CPU/GPU, use_case.

ProductResolver test: kiểm tra “mẫu đó”, “con thứ hai”, SKU và tên model.

Verifier test: kiểm tra missing field, unsupported claim, constraint mismatch và answer-cards mismatch.

Harness runtime test: kiểm tra phase, recovery, terminal event và state transition.

4.7.3. Kiểm thử frontend build

Frontend cần được build để kiểm tra TypeScript contract, import component, syntax JSX/TSX và lỗi bundling. Nếu build thành công, có thể xem đây là kiểm thử cơ bản cho khả năng đóng gói giao diện.

```
cd frontend
npm run build
```

Kết quả build thành công chứng minh các component như ProductGrid, ProductCard, CopilotDrawer, InlineProductCard và store frontend có thể biên dịch thành bundle. Tuy nhiên, build không thay thế kiểm thử tương tác thực tế; do đó vẫn cần chụp ảnh giao diện và test thủ công các kịch bản người dùng.

4.7.4. Kiểm thử kịch bản hội thoại

Các kịch bản hội thoại được thiết kế để kiểm tra đúng điểm mạnh của AURA: tư vấn theo catalog, giữ ngữ cảnh, kiểm chứng và đồng bộ product cards. Một số kịch bản nên dùng trong demo và báo cáo gồm:

Kịch bản tìm kiếm mới: “Cho tôi laptop Dell dưới 20 triệu để học lập trình”.

Hệ thống phải trả sản phẩm phù hợp ngân sách/thương hiệu và hiển thị product cards.

Kịch bản hỏi tiếp: sau khi có sản phẩm, hỏi “mẫu đó RAM bao nhiêu?”.

Hệ thống phải resolve đúng active_product_code hoặc candidate_codes.

Kịch bản so sánh: “So sánh hai mẫu vừa tư vấn”.

Hệ thống phải dùng các sản phẩm đã hiển thị, không tự chọn sản phẩm khác nếu không cần.

Kịch bản thiếu dữ liệu: hỏi về pin/trọng lượng nếu catalog thiếu field.

Hệ thống phải công bố thiếu dữ liệu, không bịa thông tin.

Kịch bản mismatch: câu trả lời không được nhắc sản phẩm ngoài `related_products` hoặc sản phẩm không tồn tại trong catalog.

Khi demo, nên chọn một kịch bản xuyên suốt và chụp ba ảnh: câu hỏi người dùng, câu trả lời kèm product cards, và nếu bật trace thì response/trace. Bộ ảnh này giúp hội đồng thấy rõ luồng từ nhu cầu tự nhiên -> QueryFrame -> catalog -> evidence -> verifier -> UI.

4.8. Đánh giá kết quả cài đặt

Sau quá trình cài đặt, hệ thống đạt được các mục tiêu chính ở mức prototype. Người dùng có thể duyệt catalog, lọc sản phẩm, xem product cards và mở AI Advisor để hỏi bằng tiếng Việt. Backend cung cấp các API cần thiết cho frontend và AI workflow. CatalogService đọc dữ liệu sản phẩm, chuẩn hóa giá/specs, hỗ trợ search và resolve sản phẩm. AI Advisor đã được đặt trong harness có các pha xử lý, state, QueryFrame, evidence, verifier, response contract và trace.

Về mặt đồ án, điểm đạt được quan trọng nhất là hệ thống không chỉ sinh câu trả lời dạng văn bản, mà trả về response có cấu trúc để frontend hiển thị sản phẩm liên quan. Đây là điều kiện cần để AI Advisor trở thành một thành phần của e-commerce thay vì chatbot rời rạc.

4.9. Kết luận chương

Chương 4 đã trình bày quá trình cài đặt và kiểm thử hệ thống AURA AI Sales Advisor theo cấu trúc của một báo cáo đồ án tốt nghiệp: môi trường triển khai, cấu trúc mã nguồn, backend API, dữ liệu catalog, frontend giao diện, AI Advisor harness và kiểm thử. Kết quả cài đặt cho thấy AURA không chỉ là một giao diện e-commerce có chatbot, mà là một hệ thống tư vấn sản phẩm có kiểm soát bằng catalog, evidence, verifier, response contract và trace.

Những nội dung trong chương này là cơ sở để Chương 5 đánh giá kết quả đạt được, chỉ ra hạn chế và đề xuất hướng phát triển. Để bản báo cáo hoàn thiện hơn, sinh viên cần thay các hình placeholder bằng screenshot thật lấy từ hệ thống đang chạy và bổ sung kết quả test thực tế từ terminal.

CHƯƠNG 5. KẾT LUẬN

Sau quá trình nghiên cứu, phân tích, thiết kế và cài đặt hệ thống từ các chương trước, chương này tổng kết các kết quả đạt được của đề án AURA AI Sales Advisor, đánh giá ưu điểm, chỉ ra hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai. Nội dung chương được trình bày theo cấu trúc thường dùng trong báo cáo đề án: đánh giá kết quả đạt được, nêu ưu điểm, phân tích hạn chế, định hướng phát triển và kết luận chương.

Khác với một báo cáo chỉ liệt kê chức năng đã xây dựng, chương này nhấn mạnh mức độ đáp ứng mục tiêu ban đầu của đề tài. Trọng tâm đánh giá không chỉ là hệ thống có giao diện catalog và API sản phẩm, mà còn là việc AI Advisor được kiểm soát bằng harness engineering để giảm rủi ro trả lời tự do, bám dữ liệu catalog, đồng bộ với product cards và có thể truy vết qua trace.

5.1. Đánh giá kết quả đạt được

Mục tiêu ban đầu của đề tài là xây dựng một hệ thống thương mại điện tử cho sản phẩm công nghệ, tích hợp AI Advisor có khả năng tư vấn bằng tiếng Việt tự nhiên và đảm bảo câu trả lời dựa trên dữ liệu catalog. Qua quá trình triển khai, hệ thống đã hoàn thành được các thành phần cốt lõi ở mức prototype, đủ để chứng minh hướng tiếp cận kỹ thuật của đề tài.

5.1.1. Kết quả về storefront và catalog sản phẩm

Hệ thống đã xây dựng được phần giao diện catalog để người dùng duyệt sản phẩm công nghệ. Các sản phẩm được hiển thị dưới dạng product card, gồm các thông tin cơ bản như tên sản phẩm, mã sản phẩm, thương hiệu, giá, ảnh và một số thông số nổi bật. Người dùng có thể xem danh sách sản phẩm, lọc theo danh mục, khoảng giá, thương hiệu và nhu cầu sử dụng.

Ở phía frontend, ProductGrid chịu trách nhiệm tổ chức danh sách sản phẩm, bộ lọc và sắp xếp. ProductCard hiển thị thông tin từng sản phẩm. Cách tách giao diện thành các component giúp mã nguồn dễ bảo trì và phù hợp với cách xây dựng ứng dụng React hiện đại. Ở phía backend, API sản phẩm cho phép lấy danh sách sản phẩm, lấy chi tiết sản phẩm theo mã và lấy ảnh sản phẩm. Nhờ đó, frontend không phụ thuộc trực tiếp vào file dữ liệu mà giao tiếp thông qua API.

Kết quả này đáp ứng mục tiêu xây dựng nền e-commerce cho đề án. Tuy nhiên, phần catalog hiện vẫn ở mức prototype, chủ yếu phục vụ hiển thị và tư vấn sản phẩm công nghệ. Các nghiệp vụ thương mại điện tử đầy đủ như giỏ hàng, thanh toán, vận chuyển hoặc quản trị kho chưa phải trọng tâm chính của đề tài.

5.1.2. Kết quả về tích hợp AI Advisor

Hệ thống đã tích hợp AI Advisor dưới dạng CopilotDrawer trên giao diện người dùng. Người dùng có thể đặt câu hỏi bằng tiếng Việt tự nhiên, chẳng hạn như hỏi laptop theo ngân sách, hỏi sản phẩm phù hợp với mục đích học tập/văn phòng/gaming, hỏi chi tiết một sản phẩm hoặc hỏi tiếp dựa trên ngữ cảnh hội thoại trước đó.

AI Advisor được kết nối với backend thông qua endpoint `/api/chat`. Request gửi lên backend chứa nội dung câu hỏi, lịch sử hội thoại và `conversation_state` nếu có. Response trả về không chỉ gồm văn bản trả lời, mà còn có các trường có cấu trúc như `answer_text`, `related_products`, `missing_fields`, `verification`, `active_context`, `conversation_state` và `decision_trace` khi được bật trong chế độ debug.

Điểm đạt được quan trọng ở phần này là AI Advisor không hoạt động như một hộp chat rời rạc. Nó được gắn trực tiếp với catalog và product cards. Khi hệ thống tư vấn một sản phẩm, frontend có thể hiển thị các sản phẩm liên quan ngay trong khung chat để người dùng đối chiếu.

5.1.3. Kết quả về harness kiểm soát AI

Đóng góp kỹ thuật nổi bật của đồ án là thiết kế lớp harness bao quanh AI Advisor. Lớp này giúp hệ thống kiểm soát quá trình xử lý của AI thay vì để mô hình ngôn ngữ trả lời trực tiếp. Câu hỏi của người dùng được đi qua các bước như xác định intent, quản lý AgentState/ConversationState, xây dựng QueryFrame, resolve tham chiếu sản phẩm, truy xuất catalog, xây dựng evidence, tạo câu trả lời, kiểm chứng bằng verifier và đóng gói response contract.

QueryFrame đóng vai trò chuyển câu hỏi tự nhiên thành các ràng buộc có cấu trúc. Ví dụ, câu hỏi “Cho tôi laptop Dell dưới 20 triệu để học lập trình” có thể được chuyển thành các trường như `category`, `brand`, `budget_max` và `use_case`. Cách làm này giúp hệ thống kết nối nhu cầu tự nhiên của người dùng với cơ chế tìm kiếm/lọc sản phẩm trong catalog.

Evidence Ledger và Verifier giúp kiểm soát chất lượng câu trả lời. Evidence Ledger ghi nhận những dữ kiện từ catalog mà hệ thống được phép sử dụng, còn Verifier kiểm tra các lỗi như sản phẩm không tồn tại, claim thiếu evidence, thiếu trường dữ liệu nhưng không công bố, constraint mismatch hoặc text trả lời không khớp product cards. Nhờ đó, AI Advisor có cơ chế an toàn hơn so với chatbot tự do.

5.1.4. Kết quả về kiểm thử và đánh giá

Đồ án đã có các nhóm kiểm thử trọng tâm cho những thành phần liên quan đến harness, verifier, query frame, product reference resolution và API contract. Các lệnh kiểm thử được tổ chức bằng pytest, phù hợp với backend Python/FastAPI. Theo ghi nhận trong báo cáo hiện tại, nhóm test trọng tâm đã có 34 test passed trong thời gian chạy ngắn, thể hiện các thành phần lỗi đã có cơ sở kiểm chứng ban đầu.

Tuy nhiên, kết quả kiểm thử cần được diễn giải thận trọng. Nhóm test hiện có chứng minh được một số hành vi kỹ thuật quan trọng của hệ thống, nhưng chưa đủ để kết luận hệ thống đã đạt chất lượng production. Để tăng độ tin cậy cho bản nộp cuối, cần bổ sung ảnh terminal chạy test, transcript hội thoại demo, ảnh response API và nếu có thể là trace của một vài tình huống tiêu biểu.

5.2. Ưu điểm của hệ thống

Từ các kết quả đã đạt được, có thể rút ra một số ưu điểm chính của hệ thống AURA AI Sales Advisor như sau:

Kiến trúc tách lớp rõ ràng: Hệ thống được chia thành frontend, backend API, catalog service và AI Advisor harness. Cách tổ chức này giúp từng phần có trách nhiệm riêng, dễ bảo trì và dễ mở rộng.

AI Advisor gắn với dữ liệu nghiệp vụ: Câu trả lời tư vấn không chỉ dựa vào khả năng sinh văn bản của mô hình, mà được neo vào catalog sản phẩm thật. Điều này làm tăng tính kiểm chứng trong bối cảnh tư vấn mua sắm.

Có cơ chế kiểm soát hành vi AI: Harness, QueryFrame, Evidence Ledger, Verifier và Response Contract tạo thành chuỗi kiểm soát trước khi câu trả lời được hiển thị cho người dùng.

Đồng bộ giữa backend và frontend: Response contract cho phép backend trả về `answer_text`, `related_products`, `display_specs`, `missing_fields` và `verification` để frontend render product cards đúng với nội dung tư vấn.

Hỗ trợ hội thoại theo ngữ cảnh: ConversationState lưu lại danh mục, ngân sách, sản phẩm đang tập trung và các candidate gần nhất, giúp hệ thống hiểu các câu hỏi tiếp nối như “mẫu đỏ” hoặc “con thứ hai”.

Có nền tảng quan sát và kiểm thử: Các endpoint health, metrics, trace khi bật debug và bộ test bằng pytest giúp hệ thống có khả năng kiểm tra, debug và đánh giá thay vì chỉ chạy demo bằng cảm tính.

Những ưu điểm trên cho thấy AURA không chỉ là một giao diện bán hàng có thêm chatbot, mà là một thử nghiệm có định hướng về cách đưa AI agent vào hệ thống phần mềm nghiệp vụ có dữ liệu, API, UI và tiêu chí kiểm chứng rõ ràng.

5.3. Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế. Việc trình bày rõ hạn chế là cần thiết để giữ tính trung thực học thuật của đề án và tránh khẳng định vượt quá phạm vi prototype.

Hạn chế về dữ liệu catalog: Catalog hiện còn giới hạn về quy mô và độ đầy đủ của trường dữ liệu. Một số thông tin như pin thực tế, trọng lượng, độ bền, benchmark, chính sách bảo hành hoặc trải nghiệm sử dụng dài hạn có thể còn thiếu hoặc chưa đồng nhất giữa các sản phẩm.

Hạn chế về mức độ nghiệp vụ e-commerce: Đề tài tập trung vào catalog và tư vấn sản phẩm, chưa chứng minh đầy đủ các nghiệp vụ production như giỏ hàng, đặt hàng, thanh toán, vận chuyển, quản trị kho hoặc quản lý người dùng nâng cao.

Hạn chế về mô hình AI: Dù đã có verifier và guardrails, LLM vẫn có rủi ro hallucination nếu prompt, retrieval, evidence hoặc postflight verification chưa bao phủ hết các tình huống. Hệ thống vì vậy cần tiếp tục mở rộng bộ kiểm chứng.

Hạn chế về kiểm thử và evaluation: Bộ test hiện có mới là nhóm test trọng tâm. Hệ thống chưa có full test suite, browser QA, load test, user study hoặc benchmark lớn để đánh giá toàn diện chất lượng tư vấn trong nhiều tình huống thực tế.

Hạn chế về triển khai production: Hệ thống chưa có đầy đủ các cơ chế vận hành như xác thực người dùng, phân quyền, rate limiting, monitoring dashboard, logging tập trung, caching production, CI/CD và kiểm thử chịu tải.

Hạn chế về minh chứng trong báo cáo: Bản nộp cuối cần bổ sung screenshot giao diện thật, transcript hội thoại, ảnh API response, log test và decision trace. Nếu còn placeholder ảnh, báo cáo sẽ yếu khi hội đồng yêu cầu bằng chứng trực quan.

Những hạn chế này không phủ nhận kết quả của đề án, mà xác định rõ phạm vi hiện tại của hệ thống: AURA là prototype tập trung vào catalog, AI Advisor và harness kiểm soát AI. Việc không khẳng định hệ thống đã sẵn sàng production giúp báo cáo có tính phản biện và đáng tin cậy hơn.

5.4. Hướng phát triển trong tương lai

Từ các hạn chế đã nêu, hệ thống có thể được tiếp tục phát triển theo một số hướng sau:

Mở rộng và chuẩn hóa catalog: Bổ sung pipeline ingestion từ nhiều nguồn, chuẩn hóa schema sản phẩm, lưu provenance theo từng field và đánh dấu độ tin cậy của từng thông số. Điều này giúp AI Advisor biết rõ dữ liệu nào có thể dùng và dữ liệu nào cần cảnh báo thiếu.

Chuyển từ CSV sang cơ sở dữ liệu production: Thiết kế và triển khai database quan hệ cho Product, Brand, Category, ProductSpec, ProductImage, EvidenceFact, ConversationSession, ChatMessage, AdvisorRun, TraceEvent và VerifierIssue. Khi dữ liệu lớn hơn, có thể kết hợp thêm search engine hoặc vector index.

Nâng cấp retrieval và ranking: Bổ sung hybrid retrieval kết hợp lọc có cấu trúc, full-text search, vector search và ranking theo use case. Nhờ đó, hệ thống có thể tư vấn tốt hơn với các câu hỏi mơ hồ như “máy học lập trình ỏn” hoặc “laptop mỏng nhẹ cho văn phòng”.

Hoàn thiện dashboard trace và evaluation: Xây dựng giao diện quản trị để xem decision_trace, phase event, verifier issue, missing fields và thời gian xử lý. Dashboard này giúp developer đánh giá agent theo toàn bộ trajectory thay vì chỉ đọc câu trả lời cuối cùng.

Mở rộng kiểm thử và user study: Tổ chức bộ dữ liệu câu hỏi đánh giá gồm search, follow-up, so sánh, hỏi field thiếu, hỏi theo ngân sách và tình huống nhập mơ hồ. Bên cạnh đó, có thể tiến hành thử nghiệm người dùng để đo thời gian tìm sản phẩm, độ hài lòng và mức độ tin tưởng vào câu trả lời.

Tăng cường bảo mật và vận hành: Bổ sung xác thực, phân quyền, rate limiting, kiểm soát API key, logging tập trung, monitoring, caching, CI/CD và load test. Đây là các điều kiện cần nếu hệ thống muốn tiến gần tới môi trường triển khai thực tế.

Cá nhân hóa tư vấn: Khi có dữ liệu người dùng hợp lệ và được xử lý đúng quy định bảo mật, hệ thống có thể học từ lịch sử xem sản phẩm, phản hồi người dùng và preference để đề xuất sản phẩm phù hợp hơn.

Các hướng phát triển trên đều xuất phát từ giới hạn hiện tại của hệ thống. Trong đó, hai hướng quan trọng nhất đối với giá trị kỹ thuật của đề tài là mở rộng dữ liệu có provenance và chuẩn hóa trace-level evaluation. Đây là hai yếu tố giúp AI Advisor trở nên đáng tin cậy hơn khi quy mô sản phẩm và độ phức tạp hội thoại tăng lên.

5.5. Kết luận chương

Chương V đã tổng kết những kết quả chính của đề tài AURA AI Sales Advisor. Hệ thống đã xây dựng được nền tảng e-commerce ở mức prototype, bao gồm catalog sản phẩm, giao diện product cards, backend API và khung tương tác AI Advisor. Bên cạnh đó, đồ án đã triển khai hướng tiếp cận harness engineering để kiểm soát AI Advisor thông qua state, QueryFrame, retrieval, evidence, verifier, response contract và trace.

Kết quả hiện tại cho thấy phần lõi của hệ thống đã đáp ứng được mục tiêu nghiên cứu/kỹ thuật ban đầu: tư vấn sản phẩm công nghệ bằng AI nhưng vẫn bám dữ liệu catalog và có cơ chế kiểm chứng. Đây là điểm khác biệt của AURA so với chatbot hoặc RAG thông thường, vì hệ thống không chỉ quan tâm đến câu trả lời cuối cùng mà còn quan tâm đến toàn bộ quá trình tạo ra câu trả lời.

TÀI LIỆU THAM KHẢO

- [1] Nielsen Norman Group, "Faceted Search: An Effective Search Tool for Ecommerce," Nielsen Norman Group. [Online]. Available: <https://www.nngroup.com/articles/faceted-search/>. Accessed: Jun. 25, 2026.
- [2] A. Vaswani et al., "Attention Is All You Need," in Proc. NeurIPS, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>.
- [3] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Proc. NeurIPS, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>.
- [4] S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," in Proc. ICLR, 2023. [Online]. Available: <https://arxiv.org/abs/2210.03629>.
- [5] OpenAI, "Tools - OpenAI Agents SDK," OpenAI. [Online]. Available: <https://openai.github.io/openai-agents-python/tools/>. Accessed: Jun. 25, 2026.
- [6] OpenAI, "Guardrails - OpenAI Agents SDK," OpenAI. [Online]. Available: <https://openai.github.io/openai-agents-python/guardrails/>. Accessed: Jun. 25, 2026.
- [7] OpenAI, "Evaluate agent workflows," OpenAI API Documentation. [Online]. Available: <https://developers.openai.com/api/docs/guides/agent-evals>. Accessed: Jun. 25, 2026.
- [8] S. Ramírez, "FastAPI Documentation," FastAPI. [Online]. Available: <https://fastapi.tiangolo.com/>. Accessed: Jun. 25, 2026.
- [9] Pydantic Team, "Pydantic Documentation," Pydantic. [Online]. Available: <https://docs.pydantic.dev/>. Accessed: Jun. 25, 2026.
- [10] Encode OSS Ltd., "Uvicorn Documentation," Uvicorn. [Online]. Available: <https://www.uvicorn.org/>. Accessed: Jun. 25, 2026.
- [11] Meta Platforms, Inc., "React Documentation," React. [Online]. Available: <https://react.dev/>. Accessed: Jun. 25, 2026.
- [12] Vite Team, "Vite Documentation," Vite. [Online]. Available: <https://vite.dev/>. Accessed: Jun. 25, 2026.
- [13] Microsoft, "TypeScript Documentation," TypeScript. [Online]. Available: <https://www.typescriptlang.org/docs/>. Accessed: Jun. 25, 2026.
- [14] pytest Development Team, "pytest Documentation," pytest. [Online]. Available: <https://docs.pytest.org/>. Accessed: Jun. 25, 2026.
- [15] Docker Inc., "Docker Documentation," Docker Docs. [Online]. Available: <https://docs.docker.com/>. Accessed: Jun. 25, 2026.
- [16] Docker Inc., "Docker Compose Documentation," Docker Docs. [Online]. Available: <https://docs.docker.com/compose/>. Accessed: Jun. 25, 2026.

- [17] IEEE, "IEEE Reference Guide," IEEE Author Center. [Online]. Available: <https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf>. Accessed: Jun. 25, 2026.
- [18] X. Tang et al., "Agent Systems with Harness Engineering," May 18, 2026. [Online]. Available: <https://github.com/RUCAIBox/awesome-agent-harness>.
- [19] Object Management Group, "Unified Modeling Language Specification, Version 2.5.1," Object Management Group, Dec. 2017. [Online]. Available: <https://www.omg.org/spec/UML/2.5.1/>.